

Уральский федеральный университет
имени первого Президента России Б.Н. Ельцина
Институт радиоэлектроники и информационных технологий

Е. Н. АКИМОВА
В. Е. МИСИЛОВ

ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ НА СУПЕРКОМПЬЮТЕРАХ С ПОМОЩЬЮ ТЕХНОЛОГИИ OpenMP

Учебное пособие

Министерство науки и высшего образования
Российской Федерации
Уральский федеральный университет
имени первого Президента России Б. Н. Ельцина

Е. Н. Акимова, В. Е. Мисилов

Параллельные вычисления на суперкомпьютерах с помощью технологии OpenMP

Учебное пособие

Рекомендовано методическим советом
Уральского федерального университета
для студентов бакалавриата и магистратуры,
обучающихся по направлениям подготовки
09.03.01, 09.04.01 — Информатика и вычислительная техника

Екатеринбург
Издательство Уральского университета
2023

УДК 004.42:004.382.2(075.8)
ББК 32.973.я73+32.971.321.1.я73
А39

Рецензенты:
кафедра шахматного искусства и компьютерной математики Уральского государственного экономического университета (зам. заведующего кафедрой, канд. экон. наук, доц. *Е. Н. Стариков*)
д-р физ.-мат. наук, ст. науч. сотр. *А. Г. Бабенко*, заведующий отделом аппроксимации и приложений Института математики и механики им. Н. Н. Красовского УрО РАН

Научный редактор — д-р физ.-мат. наук, чл.-корр. РАН, проф. *П. С. Мартышко*, заведующий лабораторией математической геофизики Института геофизики им. Ю. П. Булашевича УрО РАН

Акимова, Елена Николаевна.
А39 Параллельные вычисления на суперкомпьютерах с помощью технологии Open MP : учебное пособие / Е. Н. Акимова, В. Е. Мисилов ; М-во науки и высшего образования РФ. — Екатеринбург : Изд-во Урал. ун-та, 2023. — 104 с.

ISBN 978-5-7996-3629-6

Учебное пособие предназначено для изучения параллельных вычислений с помощью технологии OpenMP — основного средства программирования для вычислительных систем с общей памятью. Учебное пособие включает в себя описание основных директив, функций и переменных окружения стандарта OpenMP. Приводятся примеры их применения в программах на языках C/C++.

Пособие предназначено для студентов бакалавриата и магистратуры и аспирантов, обучающихся по направлениям, связанным с математическим моделированием и информационными технологиями.

Библиогр.: 12 назв. Рис. 33. Табл. 6.

УДК 004.42:004.382.2(075.8)
ББК 32.973.я73+32.971.321.1.я73

ISBN 978-5-7996-3629-6

© Уральский федеральный университет, 2023

Оглавление

Предисловие 5
Введение 6
Раздел А. Современные вычислительные системы 9
1. Списки TOP-500, Green-500, TOP-50 10
2. Суперкомпьютеры российских университетов и научных институтов 25
2.1. МГУ имени М. В. Ломоносова: суперкомпьютер «Ломоносов-2» 25
2.2. Санкт-Петербургский политехнический университет: «Политехник — РСК Торнадо» 26
2.3. Высшая школа экономики: cHARISMa 27
2.4. Межведомственный суперкомпьютерный центр РАН: «МВС-10П» 28
2.5. Объединенный институт ядерных исследований: суперкомпьютер имени Н. Н. Говоруна 29
2.6. Нижегородский государственный университет им. Н. И. Лобачевского: суперкомпьютер «Лобачевский» 30
2.7. Южно-Уральский государственный университет (Челябинск): «РСК Торнадо ЮУрГУ» 31
2.8. Сибирский суперкомпьютерный центр, ИВМиМГ СО РАН: «НКС-1П» 32
2.9. ИММ УрО РАН (Екатеринбург): суперкомпьютер «Уран» 34
3. Классификации вычислительных систем 40
3.1. Классификация Флинна 40
3.2. Дополнения Ванга и Бриггса к классификации Флинна 43
3.3. Классификация в зависимости от структуры памяти 44
4. Архитектура многоядерных систем 47
4.1. Современные многоядерные процессоры 47
4.2. Архитектура Intel Skylake 48

4.3. Архитектура сопроцессоров Intel Xeon Phi 53
Контрольные вопросы к разделу А.....56
Раздел Б.Технология OpenMP.....59
5. Использование OpenMP в языках C и C++ 60
5.1. Концепция многопоточности в OpenMP.....60
5.2. Компиляция и запуск параллельных программ на суперкомпьютере «Уран» 62
5.3. Компиляция и запуск параллельных программ на персональном компьютере.....64
6. Переменные окружения.....67
7. Библиотечные процедуры.....69
8. Директивы OpenMP 70
8.1. Директива parallel 70
8.2. Взаимодействие потоков. Директивы critical, single, master, atomic 71
8.3. Управление доступом к данным. Опции private и shared.....72
8.4. Распределение работы между потоками. Директива for.....73
8.5. Независимые параллельные секции. Директива sections 75
8.6. Выполнение независимых заданий. Директивы task, taskwait 76
8.7. Синхронизация. Директивы barrier, ordered 77
8.8. Векторизация. Использование директив simd и declare simd 77
Контрольные вопросы к разделу Б.....79
Примеры программ к разделу Б 81
Задания для самостоятельной работы к разделу Б.....93
Площадь многоугольника 93
Поиск палиндромов.....94
Задача Дирихле для уравнения Пуассона 95
Список рекомендуемой литературы.....99

Предисловие

Учебное пособие предназначено для изучения параллельных вычислений с помощью технологии OpenMP — основного средства программирования для вычислительных систем с общей памятью. Технология OpenMP используется при составлении программ для решения прикладных задач на многоядерных процессорах.

Пособие состоит из двух разделов. В разделе А описаны современные суперкомпьютеры российских научных институтов и университетов, обобщены материалы по современным суперкомпьютерам, входящим в списки TOP-500, Green-500, TOP-50, а также приведена классификация параллельных вычислительных систем и многоядерных систем. Приводятся сведения об архитектуре многоядерных систем и основных компонентах технологии OpenMP.

В разделе Б описано использование OpenMP в языках C и C++, рассмотрены компиляция и запуск программ на суперкомпьютере «Уран» и разработка параллельных программ. Описаны основные компоненты и директивы технологии OpenMP, приведены примеры программ, даны задания для самостоятельной работы.

В результате освоения материала учебного пособия обучающийся должен:

- знать классификацию высокопроизводительных вычислительных систем; архитектуру современных многоядерных процессоров; мировые и российские суперкомпьютерные системы; компоненты технологии OpenMP; возможности современных компиляторов;
- уметь компилировать и запускать программы на кластере или персональном компьютере; использовать переменные окружения, библиотечные процедуры и директивы OpenMP; выбирать подходящие для распараллеливания участки кода;
- владеть навыками использования технологии OpenMP для распараллеливания программ на системах с общей памятью и многоядерными процессорами.

Введение

Долгое время рост тактовой частоты был основным способом повышения производительность компьютеров. Со временем такой подход стал невыгодным в силу физических ограничений элементной базы. Одной из ключевых проблем остается ограниченность скорости света. Увеличение тактовой частоты также приводит к нелинейному росту тепловыделения и потребляемой мощности, поэтому современные компьютеры требуют все более серьезных систем охлаждения. Дальнейшее увеличение размера транзисторов ограничено квантовыми эффектами.

В результате производители компьютеров перенаправили свои силы в сторону развития параллельных систем. Теоретически один процессор с тактовой частотой 10 ГГц будет равен по производительности компьютеру с 1000 процессоров с частотой 10 МГц. При этом такие «слабые» процессоры будут менее энергоемкими, более простыми и дешевыми в производстве, более компактными, а также более надежными.

Поясним сказанное на примере рис. 1. В центре рисунка (рис. 1, *б*) приведены показатели эталонного процессора, принятые за 1 для последующего сравнения.

Пусть для повышения быстродействия процессора его тактовая частота увеличена на 20 % (рис. 1, *а*), тогда производительность процессора увеличится, однако не на 20, а на 10 %, в то время как энергопотребление возрастет существенно — на 80 %. Данный пример является на самом деле очень характерным — увеличение тактовой частоты процессора приводит в большинстве случаев к значительному росту энергопотребления.

При уменьшении тактовой частоты на 20 % (рис. 1, *в*) производительность процессора уменьшится примерно на 10 % (т. е. станет равной 90 % от исходной производительности). Энергопотребление процессора соответственно уменьшится, причем достаточно значительно — до уровня примерно 60 % энергопотребления исходного процессора.

И тогда, добавив в процессор второе вычислительное ядро за счет появившихся свободных транзисторов, мы можем довести суммарные показатели процессора по энергопотреблению до уровня 120 % энергопотребления исходного процессора, а производительность — до уровня 180 %.

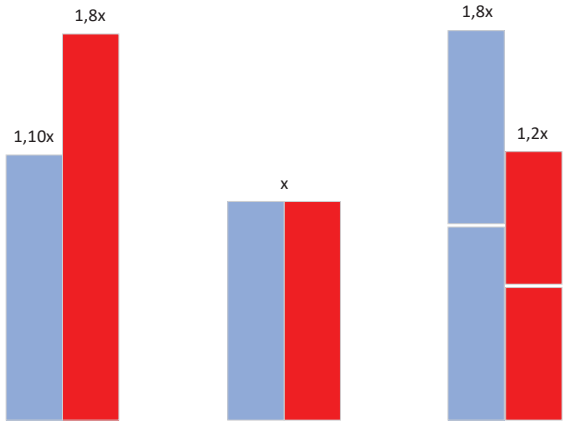


Рис. 1. Демонстрация зависимости
между энергопотреблением (правые столбцы)
и производительностью (левые столбцы) процессоров:
а — процессор с повышенной тактовой частотой; *б* — эталонный процессор;
в — двухъядерный процессор с пониженной тактовой частотой

Многоядерные системы широко распространены в мире. Это позволяет повышать производительность компьютеров, что ведет к уменьшению энергопотребления, снижению сложности логики процессоров и т. п. Поэтому многоядерность становится одним из основных направлений развития компьютерной техники. С каждым годом многоядерных систем становится все больше, их производительность растет колоссальными темпами. Согласно второму закону Мура производительность микропроцессоров удваивается каждые 18 месяцев. Сейчас практически любой компьютер оборудован многоядерным процессором. Даже настольные системы начального уровня имеют не менее двух ядер. В настоящее время типичный настольный компьютер имеет 6–8 ядер, мощные рабочие станции — 12–18.

Стоит обратить внимание, что производительность не возрастает линейно с увеличением количества ядер, то есть задействование четырех ядер не гарантирует увеличение производительности в четыре раза.

Третий способ повышения производительности состоит в добавлении дополнительных функциональных блоков. В современных процессорах обычно имеется несколько арифметико-логических устройств, что позволяет выполнять несколько независимых операций одновременно.

Разделение задач на более мелкие сегменты (увеличение количества уровней конвейера), с одной стороны, повысит производительность, потому что более мелкие операции быстрее выполняются, с другой — увеличит число прерываний, кэш-промахов и неверно спрогнозированных переходов, нарушающих нормальный ход обработки команд.

Еще один способ повысить эффективность процессора — реализовать многопоточность (Simultaneous Multithreading). Ее преимущество — возможность ввести в действие незадействованные аппаратные ресурсы. Основной принцип — одновременное исполнение нескольких программных потоков. Операционная система рассматривает многопоточный процессор как два процессора с общими кэшами и памятью. Таким образом, прикладная программа может задействовать оба таких логических процессора независимо друг от друга.

Существует проблема: при быстром росте производительности программная индустрия пока не успевает за аппаратной, и только часть приложений способна эффективно использовать ресурсы многоядерных процессоров. Каждая программа имеет один главный поток выполнения — набор инструкций, которые выполняются последовательно одна за другой. Естественно, в этом случае задействовано одно ядро процессора. Программист должен позаботиться о загрузке остальных ядер работой, иными словами, сделать так, чтобы некоторые процедуры выполнялись не последовательно, а одновременно — в параллельном режиме.

Раздел А. Современные вычислительные системы

Списки TOP-500, Green-500, TOP-50

▼

Суперкомпьютеры российских университетов и научных институтов

▼

Классификации вычислительных систем

▼

Архитектура многоядерных систем

1. Списки TOP-500, Green-500, TOP-50

Производительность вычислительных систем измеряется в Floating Point Operation per Second (Flop/s, флоп/с) — количестве выполняемых операций над дробными числами с плавающей точкой в секунду. Следует различать пиковую производительность и реальную. Пиковая производительность (R_{peak}) объявляется производителем процессоров. На практике она обычно недостижима. Реальная производительность (R_{max}) измеряется с помощью тестовых программных пакетов, реализующих реальные задачи. Одним из популярных тестов является Linpack, реализующий решение систем линейных алгебраических уравнений большого размера.

В таблице 1 перечислены пять самых мощных вычислительных систем по версии TOP-500 на 2022 год*. Список обновляется два раза в год и озвучивается на международной научной конференции ISC High Performance.

В таблице 2 приведены пять самых мощных вычислительных систем России из списка TOP-50. Кроме того, в ней перечислены вычислительные системы крупнейших научных центров России. Список составляется Научно-исследовательским вычислительным центром МГУ имени М. В. Ломоносова и Межведомственным суперкомпьютерным центром Российской академии наук. Он обновляется два раза в год на международных научных конференциях «Параллельные вычислительные технологии» и «Суперкомпьютерные дни в России».

Важной проблемой является энергопотребление современных суперкомпьютеров. Как видно из таблицы 1, наиболее мощные вычислительные системы уже потребляют десятки мегаватт. Многие из них имеют собственные электростанции. Потребляемая энергия почти полностью переходит в тепло, поэтому современные вычислительные машины также требуют высокотехнологичных систем охлаждения.

Рейтинг Green-500 составляется на основании энергоэффективности суперкомпьютеров. В таблице 3 приведены 5 самых энергоэффективных суперкомпьютеров мира.

* Ознакомиться с полным списком TOP-500 самых мощных компьютеров в мире, посмотреть динамику развития по годам можно, перейдя на сайт www.top500.org.

Таблица 1

Пять самых мощных вычислительных систем мира (ноябрь 2022)

Позиция	Местоположение	Название	Число ядер	R_{max} (Тфлоп/с)	R_{peak} (Тфлоп/с)	Энергопотребление (кВт)
1	Ок-Риджская национальная лаборатория, США	**Frontier** — HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE	8 730 112	1 102 000	1 685 650	21 100
2	Центр вычислительных наук Института физико-химических исследований (RIKEN), Япония	**Supercomputer Fugaku** — A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu	7 630 848	442 010	537 212	29 899
3	Европейское совместное предприятие по высокопроизводительным вычислениям, Финляндия	**LUMI** — HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE	2 220 288	309 100	428 700	6 016

Окончание табл. 1

Позиция	Местоположение	Название	Число ядер	R_{max} (Тфлоп/с)	R_{peak} (Тфлоп/с)	Энергопотребление (кВт)
4	Европейское совместное предприятие по высокопроизводительным вычислениям, Италия	**Leonardo** — BullSequana XH2000, Xeon Platinum 8358 32C 2.6GHz, NVIDIA A100 SXM4 64 GB, Quad-rail NVIDIA HDR100 Infiniband, Atos	1 463 616	174 700	255 750	5 610
5	Ок-Риджская национальная лаборатория,США	**Summit** — IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband, IBM	2 414 592	148 600	200 794	10 096

Таблица 2

Самые мощные вычислительные системы России (сентябрь 2022)*

№ позиции в TOP-50	Название Место установки	Узлов Проц. Ускор.	Архитектура: кол-во узлов: конфигурация узла сеть: вычислительная / сервисная / транспортная	R_{max}, R_{peak} (Тфлоп/с)	Разработчик Область применения
1	«**Червоненкис**» Яндекс, г. Москва	199 398 1592	199: CPU: 2x AMD EPYC 7702, 1024 GB RAM Acc: 8x NVIDIA A100 HDR InfiniBand / нд / 100 Gigabit Ethernet	21530.0 29415.17	**Яндекс** **NVIDIA** IT Services
2	«**Галушкин**» Яндекс, г. Москва	136 272 1088	136: CPU: 2x AMD EPYC 7702, 1024 GB RAM Acc: 8x NVIDIA A100 HDR InfiniBand / нд / 100 Gigabit Ethernet	16020.0 20636.1	**Яндекс** **NVIDIA** IT Services
3	«**Ляпунов**» Яндекс, г. Москва	137 274 1096	137: CPU: 2x AMD Epyc 7662, 512 GB RAM Acc: 8x NVIDIA A100 HDR InfiniBand / нд / 100 Gigabit Ethernet	12810.0 20029.19	**NVIDIA** **Inspur** IT Services

* Ознакомиться с полным списком TOP-50 самых мощных компьютеров России, посмотреть динамику развития по годам можно, перейдя на сайт http://top50.supercomputers.ru/list.

Продолжение табл. 2

№ позиции в TOP-50	Название Место установки	Узлов Проц. Ускор.	Архитектура: кол-во узлов: конфигурация узла сеть: вычислительная / сервисная / транспортная	R_{max}, R_{peak} (Тфлоп/с)	Разработчик Область применения
4	«**Кристофари Нео**» SberCloud (ООО «Облачные технологии»), СберБанк, г. Москва	99 198 792	99: CPU: 2x AMD EPYC 7742, 2048 GB RAM Acc: 8x NVIDIA A100 HDR InfiniBand / 10 Gigabit Ethernet / 200 Gigabit Ethernet	11 950.0 14 908.6	**NVIDIA** **SberCloud (ООО «Облачные технологии»)** Облачный провайдер
5	«**Кристофари**» SberCloud (ООО «Облачные технологии»), СберБанк, г. Москва	75 150 1200	75: NVIDIA DGX-2 CPU: 2x Intel Xeon Platinum 816824C 2.7GHz, 1536 GB RAM Acc: 16x NVIDIA Tesla V100 EDR InfiniBand / 100 Gigabit Ethernet / 10 Gigabit Ethernet	6 669.0 8 789.76	**SberCloud (ООО «Облачные технологии»)** **NVIDIA** Облачный провайдер
6	«**Ломоносов-2**» Московский государственный университет имени М. В. Ломоносова, г. Москва	1696 1696 1856	1536: CPU: 1x Intel Xeon E5-2697v3, 64 GB RAM Acc: 1x NVIDIA Tesla K40M 160: CPU: 1x Intel Xeon Gold 6126, 96 GB RAM Acc: 2x NVIDIA Tesla P100 FDR Infiniband / Gigabit Ethernet / FDR Infiniband	2 478.0 4 946.79	**Т-Платформы** Наука и образование

Продолжение табл. 2

№ позиции в TOP-50	Название Место установки	Узлов Проц. Ускор.	Архитектура: кол-во узлов: конфигурация узла сеть: вычислительная / сервисная / транспортная	R_{max}, R_{peak} (Тфлоп/с)	Разработчик Область применения
9	«**Политехник — РСК Торнадо**» Суперкомпьютерный центр, Санкт-Петербургский политехнический университет, г. Санкт-Петербург	821 1642 128	625: CPU: 2x Intel Xeon E5-2697v3, 64 GB RAM 56: CPU: 2x Intel Xeon E5-2697v3, 64 GB RAM Acc: 2x NVIDIA Tesla K40 54: CPU: 2x Intel Xeon E5-2697v3, 128 GB RAM 30: CPU: 2x Intel Xeon Platinum 8268, 192 GB RAM 2: CPU: 2x Intel Xeon Gold 6248R, 768 GB RAM Acc: 8x NVIDIA Tesla V100 51: CPU: 2x Intel Xeon Gold 6248R, 192 GB RAM 3: CPU: 2x Intel Xeon E5-2697v3, 256 GB RAM FDR Infiniband / Gigabit Ethernet / Gigabit Ethernet	971.23 1 521.27	**Группа компаний РСК** Наука и образование

Продолжение табл. 2

№ позиции в TOP-50	Название Место установки	Узлов Проц. Ускор.	Архитектура: кол-во узлов: конфигурация узла сеть: вычислительная / сервисная / транспортная	R_{max}, R_{peak} (Тфлоп/с)	Разработчик Область применения
10	**cHARISMa** Национальный исследовательский университет Высшая школа экономики, г. Москва	54 108 166	16: CPU: 2x Intel Xeon Gold 6152, 768 GB RAM Acc: 4x NVIDIA Tesla V100 10: CPU: 2x Intel Xeon Gold 6152, 1536 GB RAM Acc: 4x NVIDIA Tesla V100 6: CPU: 2x Intel Xeon Gold 6152, 768 GB RAM 2: CPU: 2x Intel Xeon Gold 6152, 768 GB RAM Acc: 1x NVIDIA Tesla P40 3: CPU: 2x Intel Xeon Gold 6240R, 768 GB RAM Acc: 4x NVIDIA Tesla V100 11: CPU: 2x Intel Xeon Gold 6248R, 384 GB RAM 6: CPU: 2x AMD EPYC 7702, 1024 GB RAM Acc: 8x NVIDIA A100 EDR Infiniband / Gigabit Ethernet / EDR Infiniband	927.4 2027.0	**Dell** **Avilex** **Hewlett Packard Enterprise** **Институт системного программирования РАН (ИСП РАН)** Наука и образование

Продолжение табл. 2

№ позиции в TOP-50	Название Место установки	Узлов Проц. Ускор.	Архитектура: кол-во узлов: конфигурация узла сеть: вычислительная / сервисная / транспортная	R_{max}, R_{peak} (Тфлоп/с)	Разработчик Область применения
11	«**МВС-10П ОП2**» Межведомственный суперкомпьютерный центр, РАН, г. Москва	249 498 н/д	58: CPU: 2x Intel Xeon Gold 6154, 192 GB RAM 51: CPU: 2x Intel Xeon Platinum 8268, 192 GB RAM 140: CPU: 2x Intel Intel Xeon Gold 6248R, 192 GB RAM Intel OmniPath / Gigabit Ethernet / Intel OmniPath	759.42 1 072.74	**Группа компаний РСК** Наука и образование
17	«**имени Н. Н. Говоруна сегмент SKYLAKE**» Лаборатория информационных технологий, Объединенный институт ядерных исследований, г. Дубна	104 208 н/д	104: CPU: 2x Intel Xeon 8268, 192 GB RAM Intel OmniPath / Fast Ethernet / Gigabit Ethernet	312.62 463.26	**Группа компаний РСК** Наука и образование

Продолжение табл. 2

№ позиции в TOP-50	Название Место установки	Узлов Проц. Ускор.	Архитектура: кол-во узлов: конфигурация узла сеть: вычислительная / сервисная / транспортная	R_{max}, R_{peak} (Тфлоп/с)	Разработчик Область применения
18	«**Лобачевский**» Нижегородский государственный университет им. Н. И. Лобачевского, г. Нижний Новгород	180 360 450	100: CPU: 2x Intel Xeon E5-2660, 65.5 GB RAM Acc: 3x NVIDIA Kepler K20X 50: CPU: 2x Intel Xeon E5-2660, 65.5 GB RAM Acc: 2x NVIDIA Fermi 2090 10: CPU: 2x Intel Xeon E5-2660, 65.5 GB RAM Acc: 2x Intel Xeon Phi 5110P 10: CPU: 2x Intel Xeon E5-2660, 65.5 GB RAM Acc: 3x NVIDIA Fermi 2090 10: CPU: 2x Intel Xeon E5-2660, 65.5 GB RAM QDR Infiniband / Gigabit Ethernet / QDR Infiniband	289.5 573.0	**Ниагара Компьютерс** Наука и образование
19	«**РСК Торнадо ЮУрГУ**» Южно-Уральский государственный университет, г. Челябинск	384 768 384	384: CPU: 2x Intel Xeon X5680, 24.6 GB RAM Acc: 1x Intel Xeon Phi SE10X QDR Infiniband / Gigabit Ethernet / QDR Infiniband	288.2 473.64	**Группа компаний РСК** Наука и образование

Продолжение табл. 2

№ позиции в TOP-50	Название Место установки	Узлов Проц. Ускор.	Архитектура: кол-во узлов: конфигурация узла сеть: вычислительная / сервисная / транспортная	R_{max}, R_{peak} (Тфлоп/с)	Разработчик Область применения
25	«**Уран**» Суперкомпьютерный центр, Институт математики и механики УрО РАН, г. Екатеринбург	76 152 394	20: CPU: 2x Intel Xeon X5675, 49.2 GB RAM Acc: 8x NVIDIA Tesla M2050 10: CPU: 2x Intel Xeon X5675, 192 GB RAM Acc: 8x NVIDIA Tesla M2090 16: CPU: 2x Intel Xeon E5-2660, 192 GB RAM Acc: 8x NVIDIA Tesla M2090 1: CPU: 2x Intel Xeon Gold 6240, 384 GB RAM Acc: 8x NVIDIA Tesla V100 SXM2 6: CPU: 2x Intel Xeon E5-2650v2, 64 GB RAM Acc: 3x NVIDIA Tesla K40M 16: CPU: 2x Intel Xeon E5-2697v4, 256 GB RAM 7: CPU: 2x Intel Xeon Gold 6254, 384 GB RAM Infiniband 4x DDR / Gigabit Ethernet / Gigabit Ethernet	194.77 326.85	**Hewlett Packard Enterprise** **Открытые технологии** Наука и образование

Окончание табл. 2

№ позиции в TOP-50	**Название Место установки**	**Узлов Проц. Ускор.**	**Архитектура: кол-во узлов: конфигурация узла сеть: вычислительная / сервисная / транспортная**	**R_{max}, R_{peak} (Тфлоп/с)**	**Разработчик Область применения**
36	«**НКС-1П**» Сибирский суперкомпьютерный центр, ИВМиМГ СО РАН, г. Новосибирск	52 88 н/д	27: CPU: 2x Intel Xeon E5–2697Av4, 128 GB RAM 16: CPU: 1x Intel Xeon Phi 7290, 112 GB RAM 1: CPU: 2x Intel Xeon Platinum 8268, 192 GB RAM 6: CPU: 2x Intel Xeon Gold 6248R, 192 GB RAM 1: CPU: 2x Intel Xeon Gold 6248R, 384 GB RAM 1:CPU: 2x Intel Xeon Gold 6248R, 768 GB RAM Intel OmniPath / Fast Ethernet / Gigabit Ethernet	120.17 181.74	**Группа компаний РСК** Исследования

Таблица 3

Пять самых энергоэффективных вычислительных систем мира (ноябрь 2022)

Позиция в Green-500	Позиция в TOP-500	Суперкомпьютер	Число ядер	R_{max} (Тфлоп/с)	Энергопотребление (кВт)	Энергоэффективность (Гфлоп/(с · Вт))
1	405	**Henri** — Lenovo ThinkSystem SR670 V2, Intel Xeon Platinum 8362 2800Mhz (32C), NVIDIA H100 80GB PCIe, Infiniband HDR, Lenovo Flatiron Institute United States	5920	2040	31	39,379
2	32	**Frontier TDS** — HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE DOE/SC/Oak Ridge National Laboratory United States	120 832	19 200	309	62,684

Продолжение табл. 3

Позиция в Green-500	Позиция в TOP-500	Суперкомпьютер	Число ядер	R_{max} (Тфлоп/с)	Энергопотребление (кВт)	Энергоэффективность (Гфлоп/(с · Вт))
3	11	**Adastra** — HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Grand Equipement National de Calcul Intensif — Centre Informatique National de l'Enseignement Suprieur (GENCI–CINES) France	319072	46 100	921	58,021
4	15	**Setonix — GPU** — HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Pawsey Supercomputing Centre, Kensington, Western Australia Australia	181248	27 160	477	56.983

Окончание табл. 3

Позиция в Green-500	Позиция в TOP-500	Суперкомпьютер	Число ядер	R_{max} (Тфлоп/с)	Энергопотребление (кВт)	Энергоэффективность (Гфлоп/(с · Вт))
5	68	**Dardel GPU** — HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE KTH — Royal Institute of Technology Sweden	52 864	8 260	146	56,491

Отметим, что самые мощные суперкомпьютеры — не обязательно самые энергоэффективные. Суперкомпьютер, занимающий наиболее высокие позиции в обоих рейтингах TOP-500 (1 место) и Green-500 (6 место), — **Frontier**, установленный в Ок-Риджской национальной лаборатории, Соединенные Штаты Америки.

2. Суперкомпьютеры российских университетов и научных институтов

В университетах и научных организациях России существуют мощные суперкомпьютеры. Приведем информацию о них более подробно.

2.1. МГУ имени М. В. Ломоносова: суперкомпьютер «Ломоносов-2»

Центр коллективного пользования сверхвысокопроизводительными вычислительными ресурсами МГУ имени М. В. Ломоносова (Суперкомпьютерный комплекс МГУ, СК МГУ, Суперкомпьютерный комплекс, ЦКП МГУ) включает в себя уникальную научную установку суперкомпьютер «Ломоносов-2» (рис. 2).



Рис. 2. Суперкомпьютер «Ломоносов-2» (г. Москва)*

Ресурсы суперкомпьютерного комплекса Московского университета предназначены для поддержки фундаментальных научных исследований и учебного процесса, а также выполнения специальных проектов, требующих привлечения суперкомпьютерных систем с петафлопсным уровнем производительности.

Суперкомпьютерный комплекс состоит из 1730 узлов, основанных на центральных процессорах Intel Xeon E5-2697 v3, Xeon Gold 6140

* Лаборатория Параллельных информационных технологий НИВЦ МГУ : [сайт]. URL: https://parallel.ru/cluster/lomonosov2.html (дата обращения: 26.10.2022).

и 6142, а также ускорителях NVIDIA Tesla K40M, P100 и V100. Суммарная производительность в тесте Linpack достигает 2,478 Пфлоп/с.

Основные направления исследований, проводимых в ЦКП МГУ:

- безопасность и противодействие терроризму;
- живые системы;
- индустрия наносистем и материалы;
- информационно-телекоммуникационные системы;
- науки о жизни;
- перспективные вооружения, военная и специальная техника;
- рациональное природопользование;
- транспортные, авиационные и космические системы;
- энергетика, энергосбережение, ядерная энергетика.

2.2. Санкт-Петербургский политехнический университет: «Политехник — РСК Торнадо»

«Политехник — РСК Торнадо» — кластерный суперкомпьютер Суперкомпьютерного центра Санкт-Петербургского политехнического университета (рис. 3). Пиковая производительность кластера 1015 Тфлоп/с. Он состоит из 668 двухпроцессорных узлов с центральными процессорами Intel Xeon E5-2697 v3. 56 узлов имеют два ускорителя вычислений NVIDIA K40.



Рис. 3. Суперкомпьютер «Политехник – РСК Торнадо» (г. Санкт-Петербург)*

* РСК : [сайт]. URL: https://clck.ru/33JHjA (дата обращения: 26.10.2022).

Среди основных исследований, проводимых в суперкомпьютерном центре «Политехнический», можно выделить:

- биохимическое моделирование;
- обработку сейсмических данных;
- исследования в области кибербезопасности.

2.3. Высшая школа экономики: cHARISMa

В Национальном исследовательском университете ВШЭ работает высокопроизводительный вычислительный кластер cHARISMa (Computer of HSE for Artificial Intelligence and Supercomputer Modelling, рис. 4).



Рис. 4. Суперкомпьютер cHARISMa (г. Москва)*

Суперкомпьютер состоит из 46 вычислительных узлов, в том числе 6 новейших узлов для обучения нейронных сетей с восьмью GPU NVIDIA A100 80 ГБ SXM в каждом, 29 специализированных узлов с большим объемом оперативной памяти 768–1536 ГБ и четырьмя графическими ускорителями NVIDIA Tesla V100 32 ГБ в каждом, а также 11 вычислительных узлов с мощными центральными процессорами. Производительность в тесте Linpack составляет 927 Тфлоп/с.

Ресурсы вычислительного кластера предназначены для поддержки проведения фундаментальных научных исследований и организации учебного процесса, а также для выполнения научных и научно-практических проектов, требующих использования суперкомпьютерных систем. Суперкомпьютер используется при выполнении исследований НИУ ВШЭ в таких областях, как:

* НИУ ВШЭ : [сайт]. URL: https://clck.ru/33KPAg (дата обращения: 26.10.2022).

- искусственный интеллект, машинное обучение;
- интеллектуальный анализ больших данных;
- математическое моделирование сложных систем и процессов;
- нейрокогнитивные исследования;
- вычислительная физика;
- прикладная математика;
- моделирование структуры и свойств химических систем и материалов;
- структурная и прикладная лингвистика;
- физика элементарных частиц и высоких энергий.

2.4. Межведомственный суперкомпьютерный центр РАН: «МВС-10П»

Межведомственный суперкомпьютерный центр Российской академии наук (МСЦ РАН) проводит исследования, связанные с созданием и использованием суперкомпьютеров и цифровой научной инфраструктуры.

МВС-10П состоит из нескольких сегментов. Новейший сегмент ОП2 (рис. 5) основан на узлах с процессорами Intel Xeon Gold 6248R, Xeon Platinum 8268 и Xeon Gold 6154. Суммарное количество узлов в сегменте — 210. Суммарная пиковая производительность — 893 Тфлоп/с.



Рис. 5. Суперкомпьютер «МВС-10П ОП2» (г. Москва)*

* Межведомственный суперкомпьютерный центр РАН : [сайт]. URL: https://www.jscc.ru/resources/hpc/ (дата обращения: 26.10.2022).

Центр осуществляет следующие фундаментальные и прикладные исследования:

- разработка и создание архитектур, системных решений, методов, прототипов, вычислительных комплексов и распределенных сред на основе гибридных и массивно-параллельных суперкомпьютерных систем;
- создание инфраструктуры для разработки программного обеспечения на базе прототипов суперкомпьютеров предельной производительности;
- разработка архитектуры, моделей, методов и средств интегрированной телекоммуникационной инфраструктуры исследовательских организаций науки и промышленности;
- исследование проблем безопасности сетевой и облачной инфраструктуры;
- организация высокопроизводительных вычислений в территориально-распределенных вычислительных средах с совместным использованием грид- и облачных технологий;
- математическое и численное моделирование задач физики плазмы, механики многофазных сред с учетом нестационарного тепломассообмена, а также в области полупроводниковых технологий;
- разработка математических моделей для задач фильтрации и массообмена в геосистемах и их адаптация к высокопроизводительным вычислительным системам;
- исследование и разработка моделей, методов и средств формирования, интеграции, сопровождения и использования электронных научно-образовательных информационных ресурсов библиотечного, архивного и музейного хранения.

2.5. Объединенный институт ядерных исследований: суперкомпьютер имени Н. Н. Говоруна

Суперкомпьютер имени Н. Н. Говоруна (рис. 6) Лаборатории информационных технологий ОИЯИ (Московская область, г. Дубна) обладает пиковой производительностью 1,7 Пфлоп/с. Новейший сегмент Skylake состоит из 104 узлов, каждый из которых имеет два процессора Intel Xeon 8268. Распределенная высокоскоростная система хране-

ния данных «PCK Storage-on-Demand» занимает ведущее место по эффективности среди суперкомпьютеров России.



Рис. 6. Суперкомпьютер им. Н. Н. Говоруна (г. Дубна)*

Ресурсы суперкомпьютера используются научными группами различных лабораторий ОИЯИ для решения широкого круга задач теоретической физики, а также для моделирования и обработки экспериментальных данных. Среди них можно выделить:

- вычисления для проекта NICA;
- задачи квантовой хромодинамики;
- расчет свойств атомов сверхтяжелых элементов;
- исследования в области радиационной биологии и радиологической безопасности.

2.6. Нижегородский государственный университет им. Н. И. Лобачевского: суперкомпьютер «Лобачевский»

Суперкомпьютер «Лобачевский» (рис. 7) Приволжского научно-образовательного центра суперкомпьютерных технологий ННГУ основан на узлах с центральными процессорами Intel Xeon E5-2660v2 и ускорителями NVIDIA Tesla K20, 2090, Intel Xeon Phi 5110P. Пиковая производительность составляет 570 Тфлоп/с.

В 2020 г. суперкомпьютер ННГУ «Лобачевский» был расширен двумя серверами с современными графическими ускорителями NVIDIA

* PCK : [сайт]. URL: https://rscgroup.ru/project/jinr-hpc-cluster-govorun/ (дата обращения: 26.10.2022).

A100, обеспечивающими высокую производительность в задачах искусственного интеллекта, анализа данных и высокопроизводительных вычислений. Каждый сервер содержит 2 процессора AMD EPYC 7742 (2,3 Тфлоп/с каждый), 512 Гб оперативной памяти, 8 графических ускорителей NVIDIA A100 (9,7 Тфлоп/с каждый, 19,5 Тфлоп/с при использовании тензорных ядер).



Рис. 7. Суперкомпьютер «Лобачевский» (г. Нижний Новгород)*

Основными направлениями научных и прикладных работ, выполняемых на базе центра суперкомпьютерных технологий, являются:

- высокопроизводительные вычисления в задачах глобальной оптимизации;
- высокопроизводительные вычисления в биомедицинских приложениях;
- технологии и программное обеспечение высокопроизводительных вычислений;
- высокопроизводительная компьютерная графика и научная визуализация.

2.7. Южно-Уральский государственный университет (Челябинск): «РСК Торнадо ЮУрГУ»

Лаборатория суперкомпьютерного моделирования Южно-Уральского государственного университета имеет инновационный энерго-

* NIAGARA Российские суперкомпьютеры : [сайт]. URL: https://www.niagara.ru/superkompyuter-lobachevskij-v-nngu/ (дата обращения: 26.10.2022).

эффективный суперкомпьютер «Торнадо ЮУрГУ», используемый для решения научных и инженерных задач (рис. 8).

Суперкомпьютер «Торнадо ЮУрГУ» состоит из 480 компактных и мощных вычислительных blade-модулей с жидкостным охлаждением всех электронных компонентов и высокой энергоэффективностью (занимал 64 место в мировом списке Green-500 и 127 место в рейтинге TOP-500 в ноябре 2013 г.). Узлы основаны на центральных процессорах Intel Xeon X5680 и сопроцессорах Intel Xeon Phi. Производительность комплекса в тесте Linpack составляет 288,2 Тфлоп/с.



Рис. 8. Суперкомпьютер «Торнадо ЮУрГУ» (г. Челябинск)*

Задачи, решаемые с помощью суперкомпьютера, включают:

- гидрогазодинамическое моделирование;
- электронное строение наносистем;
- моделирование металлургических процессов;
- краткосрочное прогнозирование погоды;
- анализ сверхбольших графов для задач дата-майнинга.

2.8. Сибирский суперкомпьютерный центр, ИВМиМГ СО РАН: «НКС-1П»

Сибирский суперкомпьютерный центр Института вычислительной математики и математической геофизики Сибирского отделения РАН оказывает вычислительные услуги 24 институтам СО РАН и 5 универ-

* Лаборатория суперкомпьютерного моделирования ЮУрГУ : [сайт]. URL: http://supercomputer.susu.ru/computers/tornado/ (дата обращения: 26.10.2022).

ситетам Сибири. Центр ориентирован на решение фундаментальных и прикладных задач в различных отраслях науки, а также подготовку специалистов в области суперкомпьютерных вычислений.

Кластер НКС-1П (рис. 9) основан на вычислительных узлах с центральными процессорами Intel Xeon E5-2697 v4 и ускорителях Xeon Phi 7290. Пиковая производительность составляет 181 Тфлоп/с.



Рис. 9. Суперкомпьютер «НКС-1П»
(г. Новосибирск)*

На суперкомпьютере выполняются исследования по различным направлениям:

- индустрия наносистем;
- информационно-телекоммуникационные системы;
- энергоэффективность, энергосбережение, ядерная энергетики;
- науки о жизни;
- рациональное природопользование;
- транспортные и космические системы;
- биология;
- вычислительная гидродинамика;
- химия;
- химические технологии;
- вычислительная математика;
- физика элементарных частиц;
- квантовая химия;
- математическое моделирование природных явлений.

* РСК : [сайт]. URL: https://clck.ru/33JKLi (дата обращения: 26.10.2022).

2.9. ИММ УрО РАН (Екатеринбург): суперкомпьютер «Уран»

ИММ УрО РАН предоставляет пользователям для работы суперкомпьютер «Уран» (рис. 10).



Рис. 10. Суперкомпьютер «Уран» (г. Екатеринбург)*

Суперкомпьютер «Уран» собран на базе blade-серверов фирмы Hewlett-Packard. Он состоит из 81 вычислительного узла, которые установлены в модулях с высокой плотностью упаковки. Вычислительные узлы оснащены процессорами Intel Xeon, работающими на частотах 2.2–3.1 ГГц, 48–384 гигабайтами оперативной памяти и графическими ускорителями NVIDIA Tesla. В общей сложности пользователям доступно 1542 вычислительных ядра CPU, 167 плат GPU и 13 Тбайт оперативной памяти. Система хранения суперкомпьютера «Уран» позволяет разместить до 140 Тбайт данных. Для передачи данных между вычислительными узлами используются высокоскоростные сети Infiniband с пропускной способностью 20 Гбит/с и 100 Гбит/с. Доступ к суперкомпьютеру «Уран» осуществляется через городскую сеть УрО РАН в Екатеринбурге по технологии 10Gi Ethernet со скоростью 10 Гбит/с.

Примеры задач, решаемых на суперкомпьютере «Уран»:

- Моделирование движения вязкой жидкости для исследования процессов тепловой конвекции в верхних слоях мантии Земли (рис. 11).
- Исследование радиационного переноса для сложных физических моделей, используемых в спектроскопической диагностике, в физи-

* Параллельные вычисления в УрО РАН : [сайт]. URL: https://parallel.uran.ru/node/3 (дата обращения: 26.10.2022).

ке плазмы, управляемом термоядерном синтезе, а также при создании лазеров, в том числе с ядерной накачкой.

• Решение актуальных проблем автономной навигации движущихся объектов по геофизическим полям:

– моделирование процессов навигации и наведения с учетом подробных моделей датчиков геофизических полей для разработки и исследования алгоритмов навигации и управления;

– разработка вычислительной технологии с использованием многоуровневого распараллеливания для конструктивной оценки информативности эталонных изображений геофизических полей.

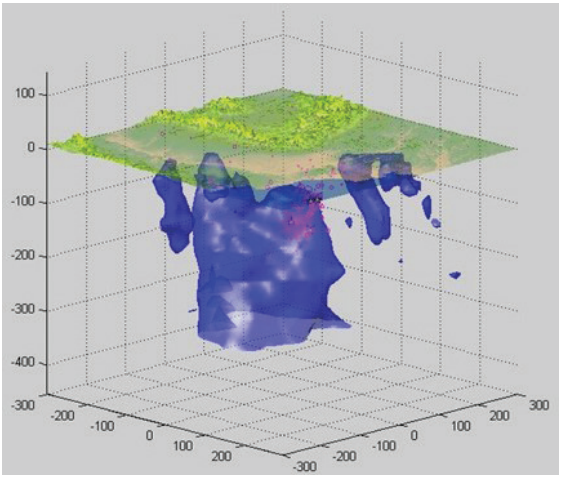


Рис. 11. Моделирование движений тектонических плит в коре Земли*

• Исследование динамики и сейсмичности различных систем тектонических плит с учетом сферической геометрии Земли и неоднородности литосферы.

• Создание технологии обработки космических изображений земной поверхности сверхбольшого размера и применение ее при топографическом дешифрировании космических снимков.

• Разработка методов и алгоритмов построения маршрутов в условиях различного типа ограничений для широкого круга задач, включая маршрутные задачи атомной энергетики.

* Моделирование погружения осколка континентальной плиты в районе юго-восточных Карпат // Параллельные вычисления в УрО РАН : [сайт]. URL: https://parallel.uran.ru/node/412 (дата обращения: 26.10.2022).

• Разработка параллельных программ для задачи взаимодействия излучения с веществом в различных физических постановках с использованием гетерогенных систем в системе параллельного программирования DVMH.

• Разработка высокоэффективных параллельных алгоритмов решения линейных и нелинейных трехмерных обратных задач гравиметрии и магнитометрии на основе итерационных процессов (рис. 12).

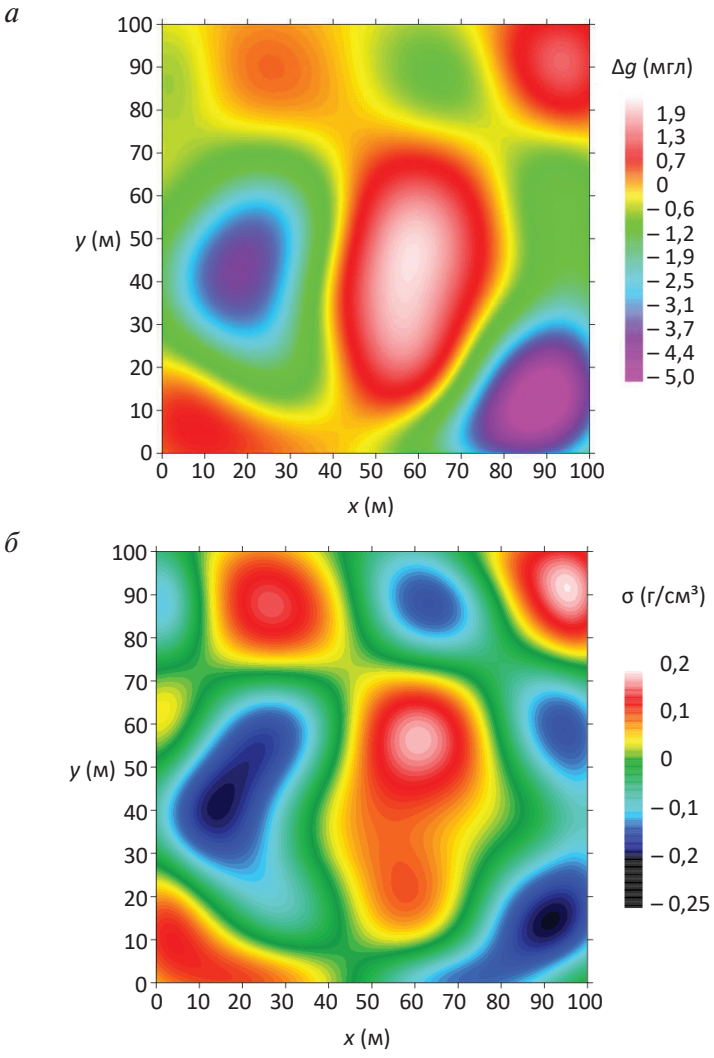


Рис. 12. Обратная задача гравиметрии:
а — гравитационное поле; *б* — искомая плотность

• Моделирование сложной динамики и оптимального управления ракетой-носителем при выведении спутников на эллиптические орбиты, в том числе на высокие геостационарные орбиты.

• Моделирование процессов тепломассопереноса в сложных многофазных средах:

– моделирование нестационарных тепловых полей в вечномерзлых почвах с учетом климатических параметров и воздействия различных технических систем (рис. 13);

– моделирование геотермальных циклических систем, состоящих из добывающих и нагнетательных скважин.

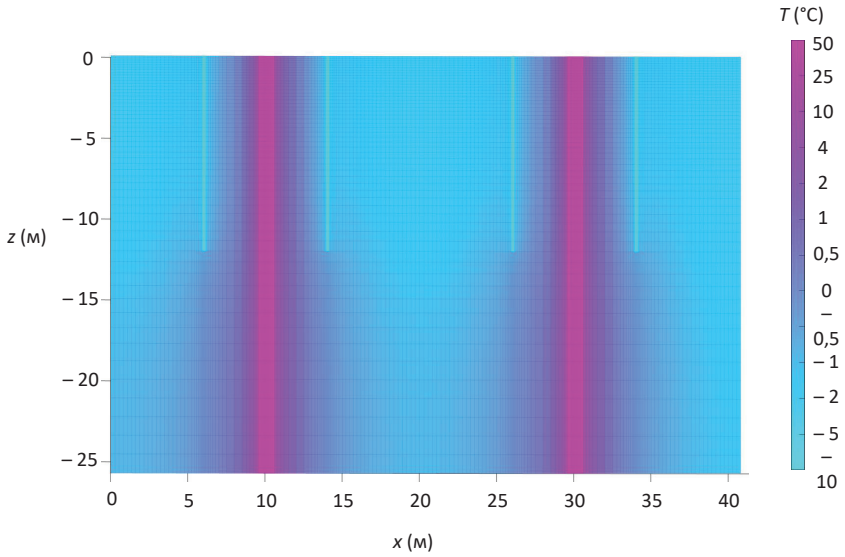


Рис. 13. Моделирование тепловых полей
в почве возле нефтяных и газовых скважин

Для проведения научных и инженерных расчетов на суперкомпьютере установлено базовое программное обеспечение:

- операционная система Linux;
- система запуска задач Slurm;
- языки программирования C, C++, Fortran;
- компиляторы Intel, GNU, PGI;
- библиотека Math Kernel Library (MKL) Intel;
- реализации MPI: OpenMPI и MVAPICH2;
- пакеты Matlab, ANSYS CFX Academic Research.

Используются узлы нескольких типов, которые образуют основные разделы (partition) кластера. Внутри разделов (кроме debug) узлы связаны между собой высокоскоростной сетью Infiniband. Узлы разных разделов связаны между собой 1 Гбит/с Ethernet.

Раздел v100

1 узел tesla-v100 (декабрь 2019):

- два 18-ядерных процессора Intel® Xeon® Gold 6240 CPU @ 2.60GHz;
- оперативная память 384 ГБ;
- 8 GPU NVIDIA Tesla V100 (32 ГБ Global Memory).

Раздел apollo6000

6 узлов apollo [17–28] (декабрь 2019/январь 2021):

- два 18-ядерных процессора Intel® Xeon® Gold 6254 CPU @ 3.10GHz;
- оперативная память 384 ГБ.

Узлы раздела *apollo6000* объединены высокоскоростной сетью Infiniband 100 Гбит/с.

Раздел apollo

16 узлов apollo [1–16] (февраль 2017):

- два 18-ядерных процессора Intel® Xeon® CPU E5-2697 v4 @ 2.30GHz;
- оперативная память 256 ГБ;
- кэш-память 45 МБ SmartCache;
- локальный жесткий диск 1 ТБ.

Узлы раздела *apollo* объединены высокоскоростной сетью Infiniband нового поколения 100 Гбит/с.

Раздел all

20 узлов tesla [1–20]:

- два 6-ядерных процессора Intel® Xeon® X5675 (3.07GHz)
- оперативная память 48 ГБ;
- кэш-память 2 x 12 MB Level 2 cache;
- локальный жесткий диск 120 ГБ.

10 узлов tesla [21–30]:

- два 6-ядерных процессора Intel® Xeon® X5675 (3.07GHz);
- оперативная память 192 ГБ;

- кэш-память 2 x 12 MB Level 2 cache;
- 8 GPU Tesla M2090 (6 ГБ Global Memory);
- локальный жесткий диск 400 ГБ.

13 узлов tesla [33–45]:

- два 8-ядерных процессора Intel® Xeon® E5-2660 (2.2 GHz);
- оперативная память 96 ГБ;
- кэш-память 2 x 20 MB Level 2 cache;
- 8 GPU Tesla M2090 (6 ГБ Global Memory);
- локальный жесткий диск 400 ГБ.

5 узлов tesla [48–52]:

- два 8-ядерных процессора Intel® Xeon® E5-2650 (2.6 GHz)
- оперативная память 64 ГБ;
- кэш-память 2 x 20 MB Level 2 cache;
- 3 GPU Tesla K40m (12 ГБ Global Memory);
- локальный жесткий диск 400 ГБ.

Узлы раздела *all* объединены высокоскоростной сетью Infiniband 20 Гбит/с.

Раздел debug

4 узла tesla [31–32, 46–47]:

- два 8-ядерных процессора Intel® Xeon® E5-2660 (2.2 GHz);
- оперативная память 96 ГБ;
- кэш-память 2 x 20 MB Level 2 cache;
- 8 GPU Tesla M2090 (6 ГБ Global Memory);
- локальный жесткий диск 400 ГБ.

Узлы раздела *debug* объединены сетью 1 Гбит/с Ethernet.

В качестве среды межузловых взаимодействий (интерконнекта) использована хорошо себя зарекомендовавшая технология Infiniband (IB). Коммутаторы в шасси ББ HP Blade Systems образуют первый уровень IB-коммутации. В качестве коммутатора IB второго уровня использован 144-портовый коммутатор Qlogic 9120.

В качестве I/O-интерконнекта использована выделенная сеть Gigabit Ethernet. Сеть имеет двухуровневую структуру. Первый уровень организован на коммутирующих модулях GbE2c Ethernet Blade Switch for HP c-Class BladeSystem, установленных в шасси ББ. К коммутирующему модулю каждого ББ внутренними гигабитными каналами подключены вычислительные модули и модуль управле-

ния. Коммутирующий модуль каждого ББ подключен к коммутатору Ethernet HP ProCurve Switch 4208v1-192. К нему подключены все ВМ, порты Host-машин, консоль управления и мониторинга.

Управляющий сервер BC-HP DL180G5 на основе процессоров Intel Xeon 5430 включает в себя:

- два четырехъядерных процессора с тактовой частотой 2,6 ГГц;
- 16 ГБ оперативной памяти;
- дисковую подсистему RAID5, состоящую из 5 дисков SATA объемом 500 ГБ каждый;
- 10/100/1000 Base-T Ethernet интерфейс;
- видеоадаптер, порт USB 2.0, порты мыши, клавиатуры.

3. Классификации вычислительных систем

3.1. Классификация Флинна

Самой ранней и наиболее известной является классификация архитектур вычислительных систем, предложенная в 1966 г. М. Флинном, которая базируется на понятии *потока*. Под потоком понимается последовательность элементов, команд или данных, обрабатываемая процессором. На основе числа потоков команд и потоков данных Флинн выделяет четыре класса архитектур: SISD, MISD, SIMD, MIMD.

1. Класс **SISD** (single instruction stream / single data stream) — одиночный поток команд и одиночный поток данных (рис. 14).

К этому классу относятся прежде всего классические последовательные машины, или иначе — машины фон-неймановского типа, например, PDP-11 или VAX 11/780. В таких машинах есть только один поток команд, все команды обрабатываются последовательно друг за другом, и каждая команда инициирует одну операцию с одним потоком данных. Не имеет значения, что для увеличения скорости обработки команд и скорости выполнения арифметических операций может применяться конвейерная обработка, — как машина CDC 6600 со скалярными функциональными устройствами, так и CDC 7600 с конвейерными попадают в этот класс.

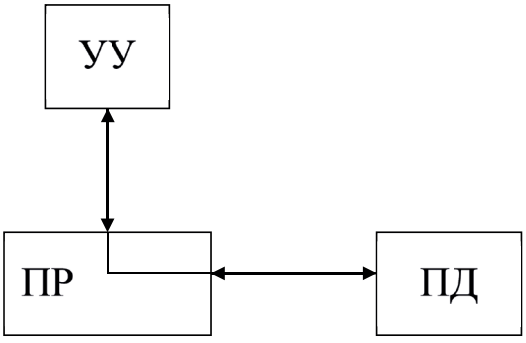


Рис. 14. Схема системы класса SISD

Многими исследователями подмечено, что в этот класс можно включить и векторно-конвейерные машины, если рассматривать вектор как одно неделимое данное для соответствующей команды. В таком случае в этот класс попадут и такие системы, как CRAY-1, CYBER 205, машины семейства FACOM VP и многие другие.

2. Класс **SIMD** (single instruction stream / multiple data stream) — одиночный поток команд и множественный поток данных (рис. 15).

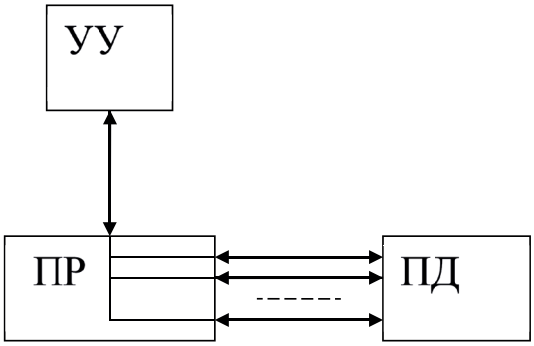


Рис. 15. Схема системы класса SIMD

В архитектурах подобного рода сохраняется один поток команд, включающий, в отличие от предыдущего класса, векторные команды. Это позволяет выполнять одну арифметическую операцию сразу над многими данными — элементами вектора. Способ выполнения векторных операций не оговаривается, поэтому обработка элементов вектора может производиться либо процессорной матрицей, как в ILLIAC IV, либо с помощью конвейера, как, например, в машине CRAY-1. Бесспорными представителями класса SIMD считаются матрицы процессоров: ILLIAC IV, ICL DAP, Goodyear Aerospace MPP, Connection Machine 1 и т. п. В таких системах единое управляющее устройство контролирует множество процессорных элементов.

Каждый процессорный элемент получает от устройства управления в каждый фиксированный момент времени одинаковую команду и выполняет ее над своими локальными данными. Классические процессорные матрицы входят в этот класс, но можно включить в него и векторно-конвейерные машины, например CRAY-1. В этом случае каждый элемент вектора надо рассматривать как отдельный элемент потока данных.

3. Класс **MISD** (multiple instruction stream / single data stream) — множественный поток команд и одиночный поток данных (рис. 16).

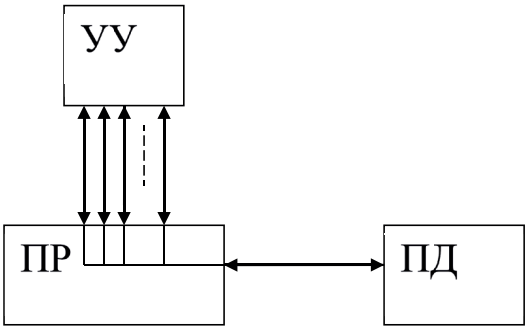


Рис. 16. Схема системы класса MISD

Определение подразумевает наличие в архитектуре многих процессоров, обрабатывающих один и тот же поток данных. Однако ни Флинн, ни другие специалисты в области архитектуры компьютеров до сих пор не смогли представить убедительный пример реально существующей вычислительной системы, построенной на этом принципе. Ряд исследователей относят конвейерные машины к этому классу, однако это не нашло окончательного признания в научном сообществе.

4. Класс **MIMD** (multiple instruction stream / multiple data stream) — множественный поток команд и множественный поток данных (рис. 17).

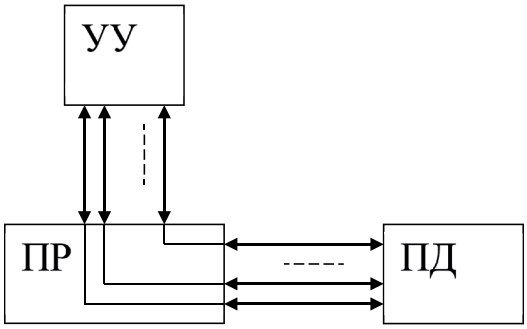


Рис. 17. Схема системы класса MIMD

Этот класс предполагает, что в вычислительной системе есть несколько устройств обработки команд, объединенных в единый комплекс и работающих каждое со своим потоком команд и данных.

Класс MIMD чрезвычайно широк, поскольку включает в себя всевозможные мультипроцессорные системы: Cm*, C.mmp, CRAY Y-MP, Denelcor HEP, BBN Butterfly, Intel Paragon, CRAY T3D и многие другие. Интересно, что если конвейерную обработку рассматривать как выполнение множества команд (операций ступеней конвейера) не над одиночным векторным потоком данных, а над множественным скалярным потоком, то все рассмотренные выше векторно-конвейерные компьютеры можно расположить и в этом классе.

Предложенная схема классификации на сегодняшний день является самой применяемой при начальной характеристике того или иного компьютера. Если говорится, что компьютер принадлежит классу SIMD или MIMD, то сразу становится понятен базовый принцип его работы, и в некоторых случаях этого бывает достаточно. Однако есть и явные недостатки. В частности, некоторые заслуживающие внимания архитектуры, например, dataflow и векторно-конвейерные машины четко не вписываются в данную классификацию. Другой недостаток — это чрезмерная заполненность класса MIMD. Необходимо средство, более избирательно систематизирующее архитектуры, которые по Флинну попадают в один класс, но совершенно различны по числу процессоров, природе и топологии связи между ними, по способу организации памяти и, конечно же, по технологии программирования.

Наличие пустого класса (MISD) не стоит считать недостатком схемы. Такие классы, по мнению некоторых исследователей в области классификации архитектур, могут стать чрезвычайно полезными для разработки принципиально новых концепций в теории и практике построения вычислительных систем.

3.2. Дополнения Ванга и Бриггса к классификации Флинна

В книге К. Ванга и Ф. Бриггса* сделаны некоторые дополнения к классификации Флинна. Оставляя четыре ранее введенных базовых класса (SISD, SIMD, MISD, MIMD), авторы внесли следующие изменения.

* Hwang, K. Computer Architecture and Parallel Processing. New York : McGraw-Hill, 1984. P. 32–40.

Класс SISD разбивается на два подкласса:

- архитектуры с единственным функциональным устройством, например PDP-11;
- архитектуры, имеющие в своем составе несколько функциональных устройств: CDC 6600, CRAY-1, FPS AP-120B, CDC Cyber 205, FACOM VP-200.

В класс SIMD также вводится два подкласса:

- архитектуры с пословно-последовательной обработкой информации: ILLIAC IV, PEPE, BSP;
- архитектуры с разрядно-последовательной обработкой: STARAN, ICL DAP.

В классе MIMD авторы различают:

- вычислительные системы со слабой связью между процессорами, к которым относят все системы с распределенной памятью, например Cosmic Cube;
- вычислительные системы с сильной связью (системы с общей памятью), куда попадают такие компьютеры, как C.mmp, BBN Butterfly, CRAY Y-MP, Denelcor HEP.

3.3. Классификация в зависимости от структуры памяти

Выделяют несколько типов построения многопроцессорных систем (рис. 18).

Симметричные мультипроцессорные системы (symmetrical multiprocessor system, SMP). В SMP системе все процессоры разделяют общую шину и общую память, могут выполнять одну и ту же задачу, причем задача может переходить от одного процессора к другому. Если один процессор отказывает, он может быть заменен другим. Необходим аппаратный протокол синхронизации кэшей всех процессоров.

Массивно-параллельная архитектура (massive parallel processing, MPP). В MPP системе память физически разделена. Система состоит из отдельных узлов содержащих процессор, локальный банк оперативной памяти, коммуникационные процессоры или сетевые адаптеры, доступ к банку оперативной памяти данного узла имеют только процессоры из этого же узла. Узлы соединяются специальными коммутационными каналами.

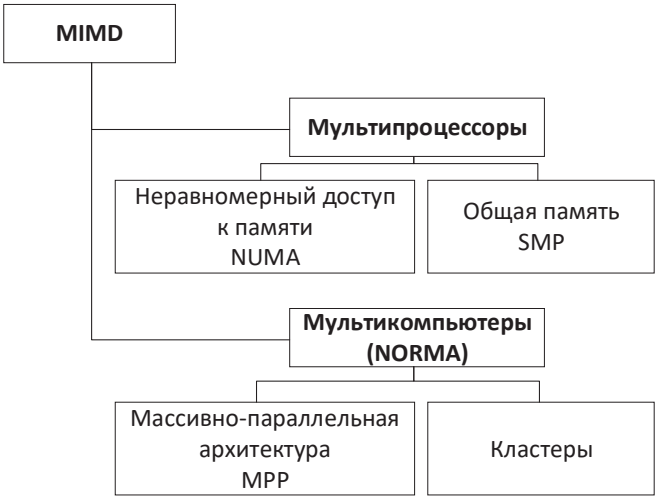


Рис. 18. Классификация многопроцессорных вычислительных систем

Следует отметить, что SMP системы входят в группу MIMD (multi instruction multi data) вычислительных систем в соответствии с классификацией Флинна. Поскольку эта классификация приводит к тому, что практически все виды параллельных систем (несмотря на их существенную разнородность) относятся к одной группе MIMD, для дальнейшей детализации класса MIMD предложена практически общепризнанная структурная схема.

В рамках этой схемы дальнейшее разделение типов многопроцессорных систем основывается на используемых способах организации оперативной памяти в этих системах. Такой поход позволяет различать два важных типа многопроцессорных систем.

Системы с общей (разделяемой, shared) памятью — процессоры могут разделять блок оперативной памяти. Преимуществами таких систем являются высокая скорость доступа к памяти и возможность использования привычной модели программирования. Недостатками — сложность технической реализации, ограничивающая масштабируемость, а также возможность конфликтов при работе с общими данными. Одна из наиболее популярных технологий программирования для таких систем — OpenMP (Open Multi-Processing).

Системы с распределенной (distributed) памятью — у каждого из процессоров своя оперативная память, к которой не может обратиться дру-

гой процессор. Это позволяет добиться высокой масштабируемости и обеспечивает отсутствие конфликтов между узлами. В то же время производительность может быть ограничена задержками и пропускной способностью сети для передачи данных между процессорами. Такие системы требуют особой модели программирования, основанной на передаче сообщений. Примером технологии, реализующей такую модель, является MPI (Message Parsing Interface).

На рис. 19 схематически представлена архитектура многоядерных систем с общей и разделяемой памятью.

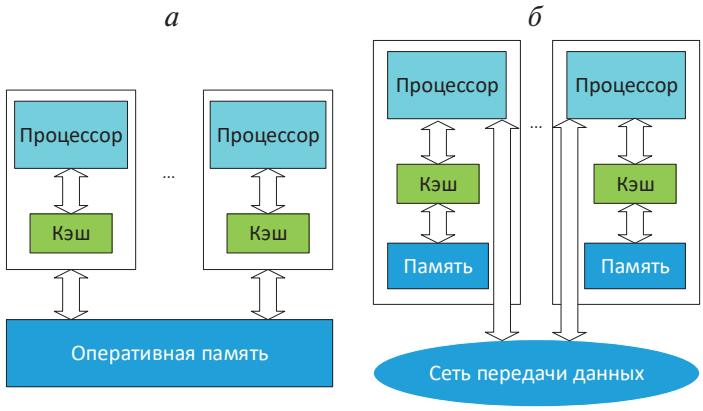


Рис. 19. Архитектура многоядерных систем:
а — с общей памятью; *б* — с распределенной памятью

SMP система является системой с общей памятью. Архитектура SMP самая дорогая с точки зрения аппаратной реализации и самая дешевая с точки зрения разработки программного обеспечения. И наоборот, MPP системы почти не требуют аппаратных затрат, но являются самым дорогим решением с точки зрения разработки ПО.

4. Архитектура многоядерных систем

4.1. Современные многоядерные процессоры

На сегодняшний день на рынке представлены множество разных типов процессоров от разных производителей. Приведем таблицу с популярными процессорами (табл. 4). Перечисленные здесь процессоры используются в игровых системах, настольных компьютерах и высокопроизводительных ноутбуках. Ознакомившись с таблицей, можно увидеть различия в количестве ядер и частоте каждого процессора.

Таблица 4

Многоядерные процессоры для настольных компьютеров, представленные на рынке России в 2023 году

Производитель	Модель процессора	Количество ядер	Максимальная частота, МГц
Intel	Core i9-13900K	24	5 800
AMD	Ryzen 9 7950X	16	5 700
Intel	Core i7-12700	12	4 900
AMD	Ryzen 7 5700X	8	4 600
Intel	Core i5-12400	6	4 400
AMD	Ryzen 5 5600	6	4 400

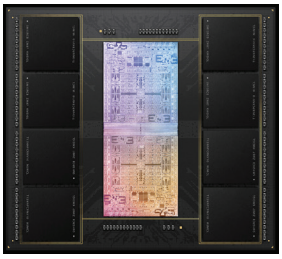
В таблице указаны далеко не все современные многоядерные процессоры, а только малая часть, представленная на рынке России.

Ниже приведем изображения некоторых многоядерных процессоров и их характеристики:



Процессор Intel Core i9-13900K

Тактовая частота: 3,0 ГГц (максимальная 5,8 ГГц).
Число ядер: 8 производительных ядер, 16 энергоэффективных ядер.
Кэш третьего уровня: 36 МБ (общий на все ядра).
Технологический процесс: Intel 7.

	**Процессор AMD Ryzen 9 7950X** Тактовая частота: 4,5 ГГц (максимальная 5,7 ГГц). Число ядер: 16. Кэш третьего уровня: 64 МБ (общий на все ядра). Технологический процесс: TSMC N5.
	**Процессор Apple M1 Ultra** Тактовая частота: 3.2 ГГц. Число ядер: 16 производительных ядер, 4 энергоэффективных ядра. Кэш третьего уровня: 96 МБ. Технологический процесс: TSMC N5.

Примечание: Сведения о процессорах приведены с сайтов: TechPowerUp : [сайт]. URL: https://clck.ru/33JkHA (дата обращения: 13.01.2023); TechPowerUp : [сайт]. URL: https://clck.ru/33JkHz (дата обращения: 13.01.2023); Apple : [сайт]. URL: https://clck.ru/33JkJz (дата обращения: 13.01.2023).

4.2. Архитектура Intel Skylake

Intel Skylake — микроархитектура процессоров x86-64 шестого поколения Intel Core для персональных компьютеров. Микропроцессоры производятся на технологическом процессе 14 нм. Первые процессоры на этой архитектуре i7-6600K (4 ядра) и i7-6700K (4 ядра с поддержкой Hyper-Threading) были выпущены в 2015 году. Процессоры Intel с шестого по десятое поколение (Skylake, Kaby Lake, Coffee Lake, Comet Lake) представляют ее развитие. Основные изменения внутри семейства состояли в увеличении тактовых частот, наращивании числа процессорных ядер (i9-10900K, выпущенный в 2020 году, имеет 10 ядер с поддержкой Hyper-Threading) и исправлении аппаратных ошибок и уязвимостей. Все процессоры этих поколений поддерживают четвертое поколение оперативной памяти DDR4 SDRAM. Основные элементы архитектуры приведены на рис. 20.

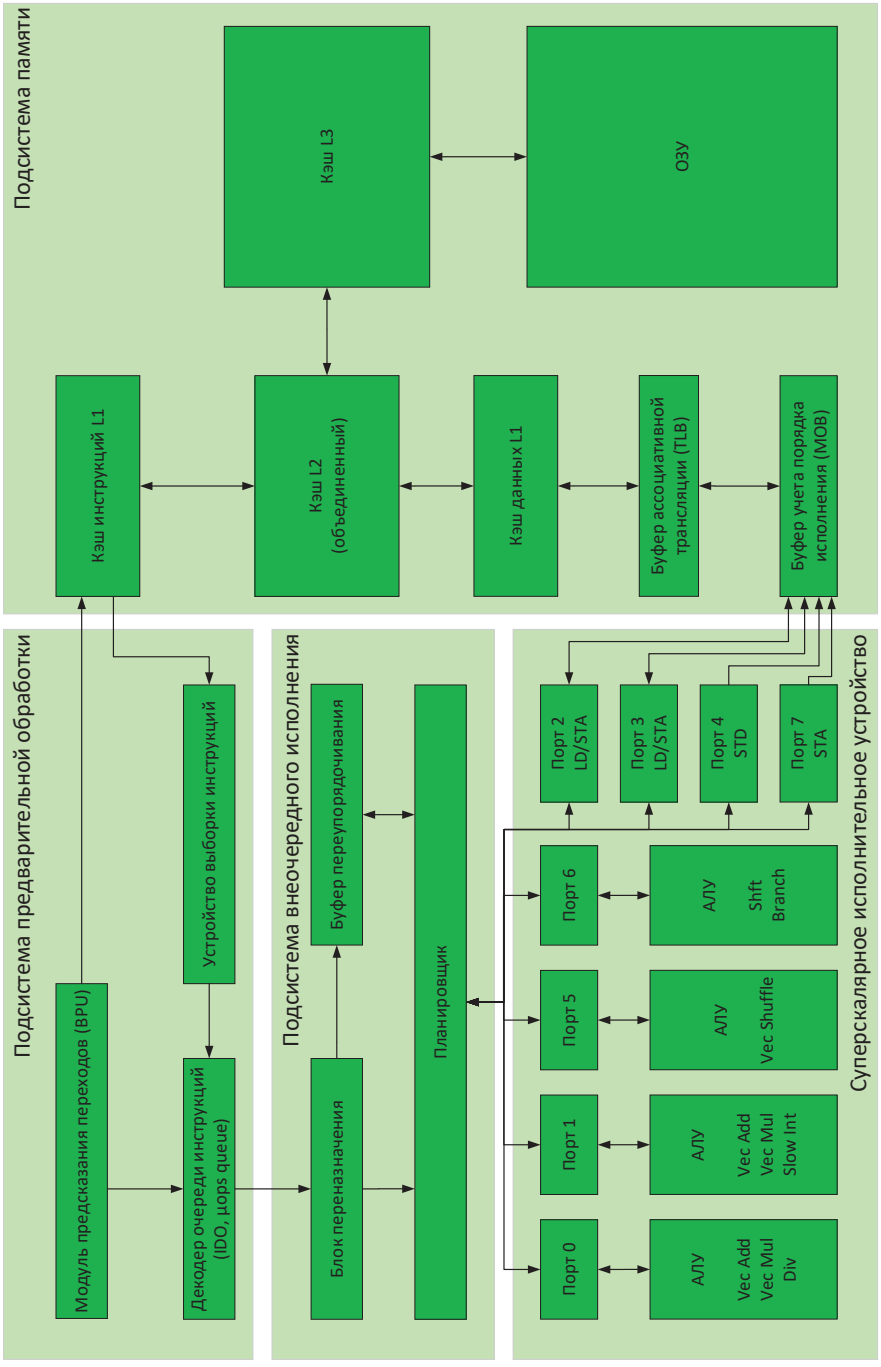


Рис. 20. Элементы архитектуры процессора x86-64

Архитектура Skylake основана на более ранних Haswell и Broadwell (4 и 5 поколения Core). По сравнению с ними были внесены следующие изменения:

- увеличены внутренние буферы для повышения эффективности конвейера и кэшей;
- улучшен предсказатель ветвлений;
- улучшена производительность операций деления;
- улучшена производительность декодера команд;
- снижено энергопотребление;
- увеличена эффективность многопоточности с использованием Hyper-Threading.

Современные процессоры архитектуры x86-64 состоят из четырех основных подсистем:

- подсистема предварительной обработки поочередного исполнения (англ. In-Order Front End) — отвечает за выборку и декодирование инструкций из кэша инструкций (Instruction Cache) в порядке, предусмотренном программой. В процессорах с многопоточностью может декодироваться несколько очередей инструкций одновременно;
- подсистема внеочередного исполнения (англ. Out-of-Order Engine) — отвечает за подготовку и размещение микроопераций в оптимальном порядке и организует взаимодействие исполнительных устройств;
- суперскалярное исполнительное устройство (англ. Execution Engine) — набор арифметико-логических устройств, исполняющих микрооперации.
- подсистема памяти (англ. Cache and Memory) — обеспечивает взаимодействие процессора с кэшем и оперативной памятью.

Рассмотрим каждую из подсистем подробнее.

Подсистема предварительной обработки поочередного исполнения

Познакомимся с устройствами этой подсистемы. К устройствам этой подсистемы относятся:

- модуль предсказания переходов (Branch Prediction Unit, BPU) – предсказывает переходы, ветвления и циклы. Для предсказания используются как динамический, так и статический методы. Последний используется в случае, если динамическое предсказание невозможно (в таблице переходов отсутствует необходимая информация);

• устройство выборки инструкций (Instruction Fetch Unit) — предзагружает инструкции в очередь инструкций. Стремится обеспечить полную загрузку очереди вне зависимости от размера инструкций и возможных ветвлений;

• декодер очереди инструкций (Instruction Decoder) — преобразует архитектурные CISC-инструкции x86-64 в оптимальные наборы RISC-микроопераций, исполняемых процессором. Способен преобразовывать несколько простых микроопераций в одну сложную, что повышает производительность и снижает энергопотребление процессора.

Подсистема внеочередного исполнения

Исполнение с изменением последовательности, при котором очередность исполнения инструкций меняется так, чтобы это не приводило к изменению результата, позволяет ускорить работу за счет более оптимального распределения запросов к исполнительным блокам и минимизации их простоев. К устройствам организации исполнения с изменением последовательности относятся:

• блок переназначения (Renamer) — передает микрооперации из системы предварительной обработки. Использует таблицу переназначения регистров (Register Alias Table) для обеспечения соответствия между регистрами архитектуры x86-64 и внутренними регистрами исполняющего устройства, используемыми при исполнении микроопераций. Такой подход позволяет выделить правильные зависимости между операциями (например, когда последующая операция требует результата предыдущей);

• буфер переупорядочивания микроопераций (Reorder Buffer) — обеспечивает выполнение микроопераций в оптимальной с точки зрения производительности последовательности. Хранит промежуточные этапы микроопераций, завершенные микрооперации и обновляет состояние процессора;

• планировщик (Scheduler, ранее — блок хранения операций, Reservation Station) хранит микрооперации, входные данные для которых еще не готовы. При первой же возможности отправляет подготовленные операции на устройства исполнения.

Порты выдачи и исполнительные устройства

Планировщик может выдавать несколько микроопераций каждый такт, по одной операции на порт выдачи. Каждый порт способен при-

нимать свой набор операций. Список соответствия портов исполнительным устройствам для архитектуры Skylake приведен в таблице 5. Список соответствия исполнительных устройств группам машинных инструкций x86-64 приведен в таблице 6.

Таблица 5

Соответствие портов исполнительным устройствам

Порт 0	Порт 1	Порты 2,3	Порт 4	Порт 5	Порт 6	Порт 7
ALU, Vec ALU	ALU, Fast LEA, Vec ALU	LD STA	STD	ALU, Fast LEA, Vec ALU	ALU, Shft	STA
Vec Shft, Vec Add	Vec Shft, Vec Add,	—	—	Vec Shuffle	Branch1	—
Vec Mul, FMA	Vec Mul, FMA	—	—	—	—	—
DIV	Slow Int	—	—	—	—	—
Branch2	Slow LEA	—	—	—	—	—

Таблица 6

Инструкции, выполняемые исполнительными устройствами

Исполнительное устройство	Количество устройств	Описание инструкций
ALU	4	Общие арифметико-логические операции
Shft	2	Операции сдвига
Slow Int	1	Сложные операции (умножение)
BM	2	Побитовые операции
Vec ALU	3	Общие арифметико-логические операции для SIMD-расширений
Vec Shft	2	Операции сдвига для SIMD-регистров
Vec ADD	2	Операции сложения для SIMD-регистров
Vec Mul/FMA	2	Операции умножения и комбинированного сложения-умножения для SIMD-регистров

Окончание табл. 6

Исполнительное устройство	Количество устройств	Описание инструкций
Vec Shuffle	1	Операции обмена элементов SIMD-регистров
DIV	1	Операции деления и вычисления квадратного корня для SIMD-регистров
LEA	2	Операции вычисления адресов памяти

Подсистема памяти

Подсистема памяти осуществляет взаимодействие с оперативной памятью. К этой подсистеме относятся:

- кэш первого уровня для данных (Level 1 Data Cache, L1d) — память с малым временем доступа (4–7 тактов), предназначенная для хранения данных объемом 32 КБ;
- кэш первого уровня для инструкций (Level 1 Instruction Cache, L1i) — память с малым временем доступа, предназначенная для хранения инструкций объемом 32 КБ;
- кэш второго уровня (Level 2 Cache, L2) — память с малым временем доступа (12–16 тактов) объемом 256 КБ;
- кэш третьего уровня (Level 3 Cache, L3) — память, общая для всех ядер процессора, время доступа от 44 тактов, объемом до 2 МБ на ядро;
- буфер ассоциативной трансляции (Translation Lookaside Buffer, TLB) — специализированный кэш, используемый для хранения адресов страниц виртуальной памяти в физической. Процессор обладает двумя уровнями TLB по аналогии с двумя уровнями кэша. Верхний уровень имеет меньший объем, но большую скорость доступа. Если адрес нужной страницы отсутствует в буфере, процессор вынужден обращаться к таблице страниц операционной системы, используя обработчика промаха страницы (Page Miss Handler, PMH);
- буфер учета порядка исполнения (Memory Ordering Buffer, MOB) — обеспечивает правильное состояние оперативной памяти (видимой программе) с учетом внеочередного исполнения инструкций загрузки и сохранения;
- блок шинного интерфейса (Bus Interface Unit) — управляет системной шиной.

4.3. Архитектура сопроцессоров Intel Xeon Phi

Xeon Phi является многоядерным сопроцессором, предназначенным для высокопроизводительных вычислений (рис. 21).

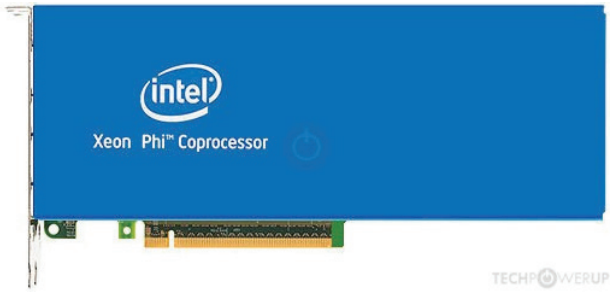


Рис. 21. Внешний вид ускорителя
Intel Xeon Phi 7120X*

Сопроцессор Intel Xeon Phi включает до 61 процессорного ядра, соединенных высокопроизводительной встроенной кольцевой шиной (рис. 22). 8 контроллеров памяти (GDDR MC) обслуживают 16 каналов GDDR5, обеспечивая суммарную пропускную способность 352 ГБ/с. Отдельный компонент (SBOX) реализует клиентскую логику PCI Express.

Исполнительное ядро выполняет 4 аппаратных потока. Поддерживается выполнение 32- и 64-битного кода, совместимого с архитектурой Intel64. Не реализованы команды расширений Intel Streaming SIMD Extensions (SSE), MMX и Advanced Vector Extensions (AVX).

Ядро сопроцессора Intel Xeon Phi содержит следующие компоненты (рис. 23).

512-битный блок векторных вычислений (vector processor unit, VPU) включает расширенный блок математических вычислений (extended math unit, EMU), способный выполнять операции с 16 числами с плавающей точкой одинарной точности. Кэши инструкций и данных (32 Кб L1 I–Cache и 32 Кб L1 D-Cache). Интерфейс кольцевой шины (Core-Ring Interface, CRI/L2) обеспечивает связь ядра с кольцевой шиной, а также включает 512 Кб кэша L2. Каталог тегов (Tag Directory, TD) обеспечивает когерентность L2-кэшей всех ядер.

* TechPowerUp : [сайт]. URL: https://www.techpowerup.com/gpu-specs/xeon-phi-7120x.c2481 (дата обращения: 26.10.2022).

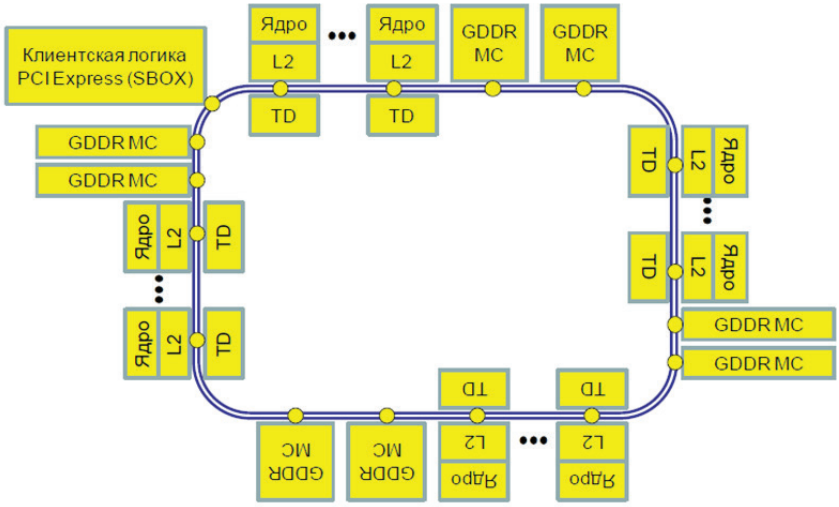


Рис. 22. Основные компоненты сопроцессора Intel Xeon Phi*

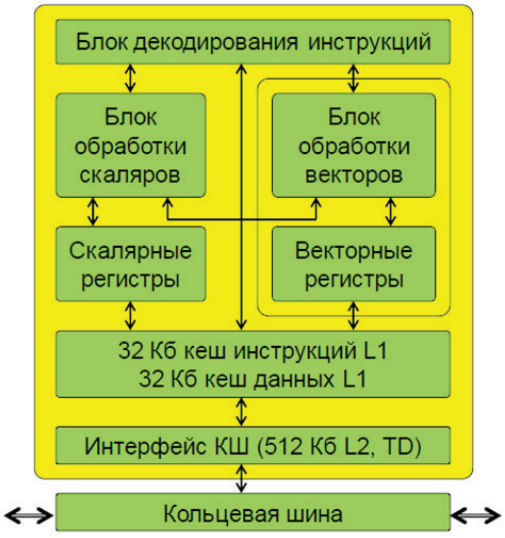


Рис. 23. Основные компоненты ядра сопроцессора Intel Xeon Phi**

* Линев А. В. Архитектура Intel Xeon Phi : Лекция № 2. Нижний Новгород : ННГУ им. Н. И. Лобачевского, 2013. С. 4. URL: Lection02.pdf (unn.ru) (дата обращения: 26.10.2022).

** Линев А. В. Архитектура Intel Xeon Phi : Лекция № 2. Нижний Новгород : ННГУ им. Н. И. Лобачевского, 2013. С. 5. URL: URL: Lection02.pdf (unn.ru) (дата обращения: 26.10.2022).

Сопроцессор Intel Xeon Phi содержит 61 ядро, но он исполняет собственную операционную систему, и одно ядро выделено для исполнения кода ОС, обслуживания прерываний и т. п. Поэтому в расчетах производительности предполагается, что для вычислений используется 60 ядер из имеющихся 61. Теоретическая производительность сопроцессора Intel Xeon Phi 7120X с 60 ядрами и частотой 1,333 ГГц составляет 2 560 Гфлоп/с для вещественных чисел одинарной точности.

Контрольные вопросы к разделу А

1. Назовите характеристики, на основе которых составляются рейтинги суперкомпьютеров.
2. Как измеряется производительность суперкомпьютеров? В чем разница между максимальной и пиковой производительностью?
3. Почему важна энергоэффективность суперкомпьютеров?
4. Перечислите основные суперкомпьютерные центры России.
5. Перечислите основные характеристики суперкомпьютера «Ломоносов-2».
6. Перечислите основные характеристики суперкомпьютера «Политехник — РСК Торнадо».
7. Перечислите основные характеристики суперкомпьютера cHARISMa.
8. Перечислите основные характеристики суперкомпьютера им. Н. Н. Говоруна.
9. Перечислите основные характеристики суперкомпьютера «Лобачевский».
10. Перечислите основные характеристики суперкомпьютера «РСК Торнадо ЮУрГУ».
11. Перечислите основные характеристики суперкомпьютера «НКС-1П».
12. Перечислите основные характеристики суперкомпьютера «Уран».
13. Назовите основные признаки классификации Флинна.
14. Приведите классы вычислительных систем по Флинну с примерами.
15. Перечислите преимущества и недостатки систем с общей и разделенной памятью.
16. Какие технологии могут использоваться для программирования систем различных классов?
17. Перечислите основные подсистемы современных центральных процессоров.
18. Какие устройства задействуются при предварительной обработке инструкций?
19. Как организована подсистема памяти?
20. Назовите основные компоненты ускорителя Xeon Phi.

Раздел Б.
Технология OpenMP

Использование OpenMP в языках C и C++

▼

Переменные окружения

▼

Библиотечные процедуры

▼

Директивы OpenMP

5. Использование OpenMP в языках C и C++

5.1. Концепция многопоточности в OpenMP

Одним из наиболее популярных средств программирования для компьютеров с общей памятью, базирующихся на традиционных языках программирования и использовании специальных управляющих модификаторов, в настоящее время является технология OpenMP. За основу берется последовательная программа, а для создания ее параллельной версии пользователю предоставляется набор директив, функций и переменных окружения. Предполагается, что создаваемая параллельная программа будет переносимой между различными компьютерами с общей памятью, поддерживающими OpenMP API.

Технология OpenMP нацелена на то, чтобы пользователь имел один вариант программы для параллельного и последовательного выполнения. Однако возможно создавать программы, которые работают корректно только в параллельном режиме или дают в последовательном режиме другой результат. Более того, из-за накопления ошибок округления и неассоциативности операций над числами с плавающей точкой результат вычислений с использованием может в некоторых случаях различаться даже при запуске одной и той же программы.

Интерфейс OpenMP задуман как стандарт для программирования на масштабируемых SMP-системах (SSMP, ccNUMA и других) в модели общей памяти (shared memory model). В стандарт OpenMP входят спецификации набора директив компилятора, вспомогательных функций и переменных среды.

OpenMP реализует параллельные вычисления с помощью многопоточности, в которой «главный» (Master) поток при входе в параллельную область создает (Fork) набор «подчиненных» (Slave) потоков, и задача распределяется между ними. Предполагается, что потоки выполняются параллельно на машине с несколькими процессорами, причем количество процессоров не обязательно должно быть больше или равно количеству потоков. После завершения параллельной области все потоки, кроме главного, завершаются (Join), и продолжается последовательная область (рис. 24). Параллельные области также могут быть вложены друг в друга.

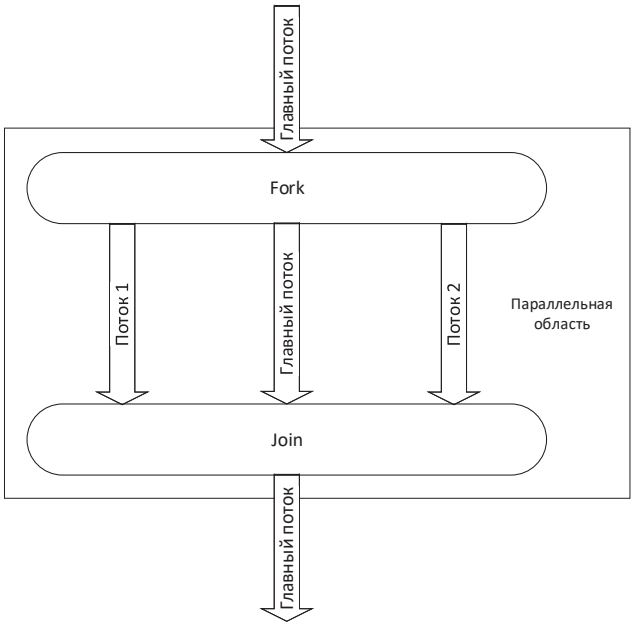


Рис. 24. Схема Fork-Join

Создание и уничтожение потоков — достаточно быстрая операция, поэтому обычно не сильно влияет на производительность программы.

OpenMP можно рассматривать как высокоуровневую надстройку над Pthreads (или аналогичными библиотеками создания потоков, например, встроенной библиотекой Thread в стандарте C++11); в OpenMP используется терминология и модель программирования, близкая к Pthreads, например, динамически порождаемые потоки, общие и разделяемые данные, механизмы блокирования для синхронизации.

Согласно терминологии POSIX threads любой UNIX-процесс состоит из нескольких потоков управления, которые имеют общее адресное пространство, но разные потоки команд и раздельные стеки. В простейшем случае процесс состоит из одного потока. Потоки иногда называют также нитями (threads), легковесными процессами, LWP (light-weight processes).

Важным достоинством технологии OpenMP является возможность реализации так называемого инкрементального программирования, когда программист постепенно находит в программе участки, содержащие ресурс параллелизма, с помощью предоставляемых механизмов делает их параллельными, а затем переходит к анализу следующих участков. Таким образом, в программе постепенно становится

все меньше нераспараллеленная часть. Такой подход значительно облегчает процесс адаптации последовательных программ к параллельным компьютерам, а также отладку и оптимизацию.

Таким образом, основными преимуществами OpenMP являются:

- простота использования,
- переносимость,
- низкие накладные расходы.

Технология OpenMP состоит из трех основных компонент, используемых программистом для составления и запуска параллельных программ:

- переменные окружения;
- встроенные библиотечные функции и процедуры;
- директивы компилятора.

5.2. Компиляция и запуск параллельных программ на суперкомпьютере «Уран»

Компиляция программ выполняется командами с добавление ключа для OpenMP.

Для компилятора ICC:

- `icc -qopenmp <исх.файл>.c -o <исполняемый файл>`
- `icpc -qopenmp <исх.файл>.cpp -o <исполняемый файл>`

Для компилятора GCC:

- `gcc -fopenmp -c <исх.файл>.c -o <исполняемый файл>`
- `g++ -fopenmp -c <исх.файл>.cpp -o <исполняемый файл>`

Для файлов на C и C++ важно расширение имени файла: для языка C это.c, для языка C++ это.cxx.

Параллельная OpenMP-программа является также корректной последовательной программой. Если скомпилировать ее без ключей OpenMP, компилятор выдаст предупреждение и воспримет код как последовательный. Директивы OpenMP будут проигнорированы.

Система запуска задач SLURM позволяет с помощью команд `sbatch` и `srun` работать в пакетном и интерактивном режимах соответственно. Пакетный режим является основным в работе с кластером «Уран».

Команда `srun` для запуска интерактивной программы имеет вид:

```
srun [опции] <исполняемый файл> [параметры программы]
```

Полный список опций можно получить, выполнив команду `man srun`. Наиболее часто используемые опции:

• `-n <число процессов>` — задает число процессов для запуска, по умолчанию n = 1. В OpenMP-программах обычно используется один процесс, поэтому данная опция редко используется.

• `-t <время>` — задает лимит времени выполнения программы, по умолчанию 30 минут.

• `-N <кол-во узлов>` — задает число узлов (nodes) для задачи, если пользователю это важно. OpenMP-программы обычно работают на одном узле.

• `-c <кол-во ядер>` (или `-cpus-per-task=<кол-во ядер>`) — задает число логических процессоров (или ядер CPU), выделяемых на один процесс.

• `--mem-per-cpu=<MB>` — задает минимальную память в расчете на одно ядро в мегабайтах; если не задано, то по умолчанию 1 ГБ.

• `--mem=<MB>` — задает общий лимит памяти на узле в мегабайтах. Эта опция исключает предыдущую.

• `-p <раздел>` или `-partition=<раздел>` — позволяет указать раздел (partition) кластера для запуска задачи. Основные разделы: *tesla*, *apollo*, *v100* и *debug*. Эти разделы покрывают весь кластер и взаимно не пересекаются, т. е. содержат разные узлы (см. главу о кластере «Уран»). Список всех разделов можно получить командой `sinfo -s`.

• `-w <имя узла>` — позволяет указать конкретный узел для запуска задачи.

Интерактивный режим подразумевает ввод и вывод в реальном времени через интерфейс командной строки (консоль, терминал). Стандартные потоки ввода, вывода и ошибок можно переадресовать в файл, используя перенаправление:

```
srun myprogram < in.txt > out.txt 2> err.txt
```

Команда `sbatch` предназначена для запуска программ в пакетном режиме:

```
sbatch [опции] --wrap="srun <имя программы> [параметры программы]"
```

или

```
sbatch [опции] <имя скрипта> [параметры скрипта]
```

Скрипт для запуска OpenMP-программы с 18 потоками на 18-ядерном процессоре может выглядеть так:

```
#!/bin/bash
srun -p apollo -c 18 myprogram
```

Описание всех опций и примеры команд можно посмотреть с помощью команды `man sbatch`.

По умолчанию стандартные потоки вывода и ошибок направляются в файл с именем `slurm-%j.out`, где `%j` заменяется уникальным идентификатором запущенной задачи. Перенаправление потоков можно выполнить, указав команде `sbatch` опции:

- `-input=<filename pattern>`
- `-output=<filename pattern>`
- `-error=<filename pattern>`

При запуске на кластере задача попадает в очередь заданий, и ей присваивается уникальный идентификатор.

Для просмотра очереди задач и информации о кластере используются следующие команды:

- `squeue` — просмотр очереди (информации о задачах, находящихся в счете или в очереди на счет);
- `sacct` — просмотр задач текущего пользователя за сутки (с начала текущего дня);
- `sinfo` — просмотр информации об узлах (прежде всего, о состоянии узлов: доступны, заняты, свободны, ...);
- `scontrol` — выдача детальной информации об узлах, разделах, задачах.
- `scancel` — отмена выполнения задачи, ожидающей старта в очереди или уже стартовавшей.

Примеры использования:

- `scancel <id задачи1> <id задачи2> ...` — убрать из очереди задачи с указанными ID;
- `scancel -u <имя пользователя>` — убрать из очереди все задачи пользователя;
- `scancel –state=PENDING -u <имя пользователя>` — убрать из очереди ожидающие запуска задачи пользователя.

5.3. Компиляция и запуск параллельных программ на персональном компьютере

Параллельные программы на языках C/C++ можно скомпилировать и запустить на персональном компьютере с системой Windows, используя среду разработки Microsoft Visual Studio. Важно помнить, что встроенный компилятор Visual Studio полностью поддерживает только стандарт OpenMP 2.0.

Для компиляции параллельных программ нужно включить опцию **OpenMP Support** в свойствах проекта на странице **Language** (рис. 25).

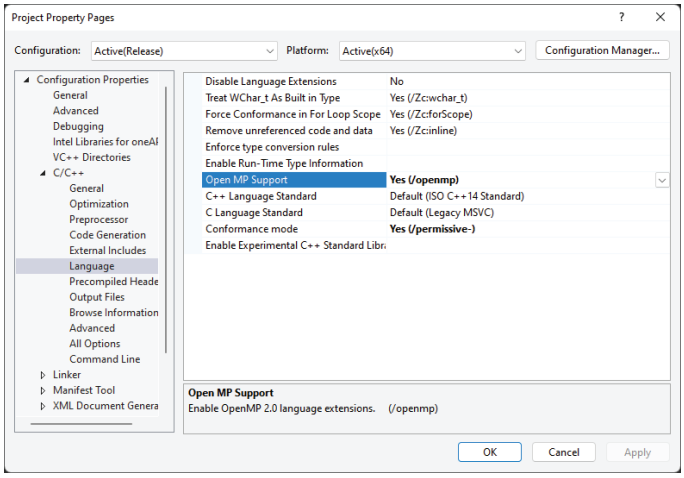


Рис. 25. Включение поддержки OpenMP
в Microsoft Visual Studio

Кроме того, можно использовать бесплатные пакеты Intel oneAPI Base Toolkit и one API HPC Toolkit, включающие компилятор Intel C++ Compiler. Данные пакеты поддерживают интеграцию в Visual Studio. Компилятор Intel полностью поддерживает стандарт OpenMP 4.5.

После установки можно переключить любой проект на компилятор Intel C++ через страницу свойств (рис. 26).

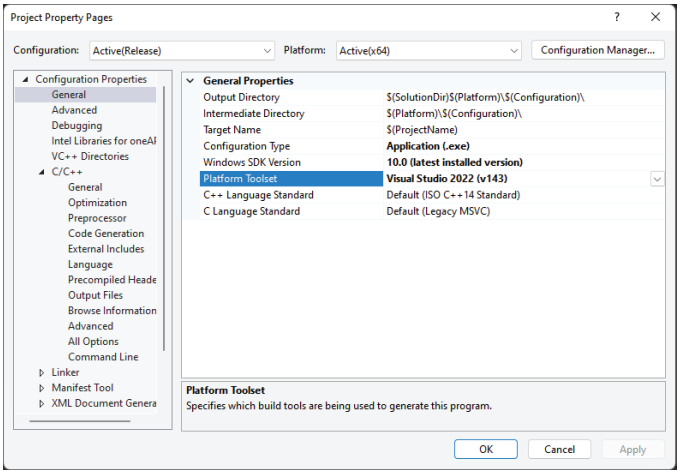


Рис. 26. Переключение используемого компилятора
в Microsoft Visual Studio

Для поддержки OpenMP нужно включить соответствующую опцию в разделе **Language [Intel C++]** (рис. 27).

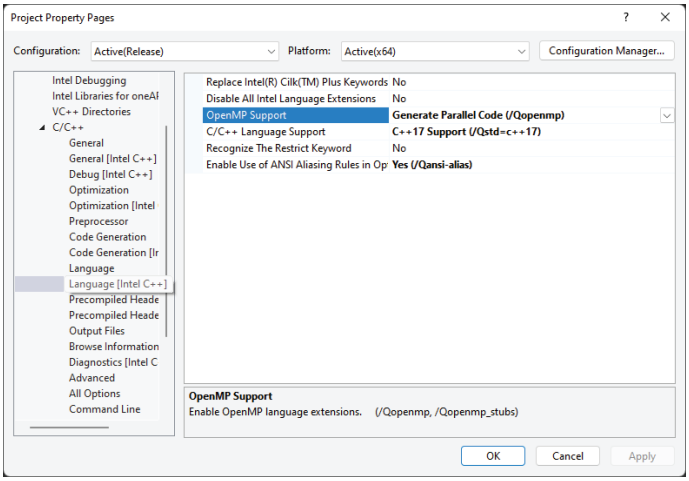


Рис. 27. Включение поддержки OpenMP
при использовании компилятора Intel в Visual Studio

Для отладки программ можно использовать вариант этой опции, включающий последовательный режим (ключ / **Qopenmp-stubs**). В таком случае компилятор проигнорирует директивы OpenMP, но программа сможет вызывать библиотечные процедуры.

6. Переменные окружения

Перед запуском программы некоторые настройки OpenMP могут быть заданы значениями переменных окружения (переменных среды, Environment Variables) операционной системы.

Наиболее часто используемой переменной окружения является `OMP_NUM_THREADS`. Она задает количество потоков, которые будут создаваться в программе для выполнения параллельных областей.

В Linux переменные среды устанавливаются командой export, например:

```
export OMP_NUM_THREADS=<число потоков>
```

В командной строке Windows используется команда set:

```
set OMP_NUM_THREADS=<число потоков>
```

Если значение переменной не задано, то количество потоков будет равным количеству логических процессоров, доступных программе. Чтобы сбросить переменную, можно присвоить ей пустое значение.

При запуске программ непосредственно из Visual Studio можно прописать нужные значения переменных окружения в настройках проекта (рис. 28).

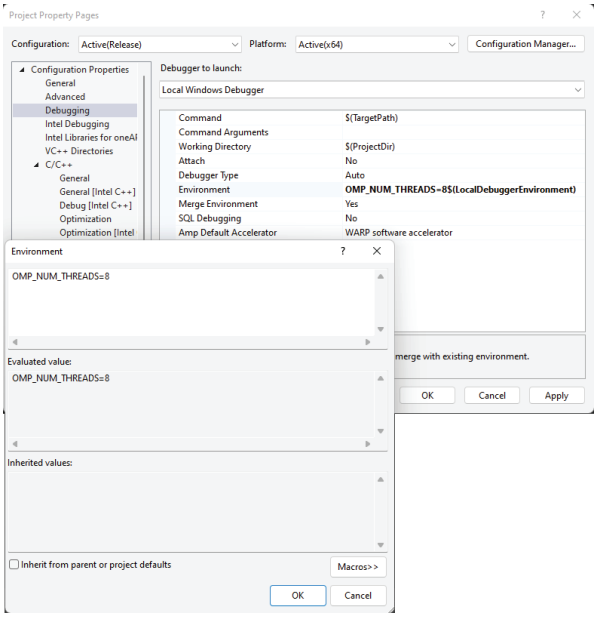


Рис. 28. Задание переменных окружения
в настройках проекта Visual Studio

Обычно потоки автоматически распределяются по процессорам операционной системой.

В стандарте OpenMP 4.0 можно явно указать, как будут распределяться потоки по логическим ядрам/физическим ядрам/процессорам, и привязать потоки к виртуальным OpenMP-процессорам (OpenMP places), используя следующие переменные:

```
OMP_PLACES = <sockets|cores|threads>
OMP_PROC_BIND = <close|spread>
```

7. Библиотечные процедуры

Чтобы использовать следующие процедуры времени выполнения (runtime) в программе на языках C/C++, нужно подключить заголовочной файл `<omp.h>`.

Команды выполняют следующие действия:

- `int omp_get_num_threads()` — возвращает количество потоков, выполняющих в данный момент текущую параллельную область;
- `int omp_get_num_procs()` — возвращает количество процессоров, доступных программе;
- `int omp_get_thread_num()` — возвращает уникальный номер потока в текущей параллельной области;
- `void omp_set_num_threads(int n)` — устанавливает число потоков, которые будут выполнять параллельные области;
- `double omp_get_wtime()` — возвращает время в секундах, прошедшее с некоторого фиксированного момента, постоянного во время существования программы. Если вызвать данную функцию перед и после некоторого участка программы, то разность значений будет равна времени выполнения данного участка.
- `double omp_get_wtick()` — возвращает длительность системного такта в секундах. Можно использовать как погрешность замеров времени.

8. Директивы OpenMP

Директивы компилятора — основной способ использования технологии OpenMP. Программист должен явно добавлять их в программу, чтобы она выполнялась в параллельном режиме.

В общем случае директива OpenMP выглядит следующим образом:
`#pragma omp <директива> [опция1] [опция2] ...`

Здесь «`#pragma`» — ключевое слово, указывающее, что конструкция является директивой препроцессора, «`omp`» — означает, что данная директива является директивой OpenMP.

Большинство директив действуют на один оператор или структурный блок, перед которым они расположены. Директивы можно разделить на три основные категории:

- создание и управление параллельными областями;
- распределение работы по потокам;
- синхронизация потоков.

8.1. Директива `parallel`

Директива указывает компилятору, что последующий структурный блок (одна строка или блок внутри фигурных скобок) является параллельной секцией. При запуске секции создаются (fork) подчиненные потоки.

После завершения работы всех потоков секции подчиненные потоки уничтожаются и управление передается следующему структурному блоку основного потока (join).

Опции:

- **if (<условие>)** — условия для создания параллельной секции. Если опция не указана, параллельная секция создается всегда.
- **num_threads (<число потоков>)** — количество потоков, создаваемых для выполнения параллельной секции. Если опция не указана, используется количество потоков по умолчанию.
- **shared (<список переменных>)** — указывается, какие переменные должны быть доступны для всех потоков.

- **private (<список переменных>)** — указывается, каким переменным нужны индивидуальные копии в каждом потоке. Переменные в подчиненных потоках будут объявлены, но не инициализированы.
- **firstprivate (<список переменных>)** — указывается, каким переменным нужны индивидуальные копии в каждом потоке. Переменные в подчиненных потоках инициализируются значениями из основного потока.
- **reduction (<операция>:<переменная>)** — указывает, что в каждом потоке должна создаваться индивидуальная копия переменной, после завершения структурного блока переменные будут объединены с использованием заданной операции. Доступные операции: +, −, *, &, |, ^, &&, ||, min, max. Переменные будут инициализированы нейтральным значением для заданной операции. (Операции min и max *не поддерживаются* в OpenMP 2.0. Кроме того, в 2.0 возможна редукция только единичных переменных, в более новых стандартах можно работать с массивами).
- **default (shared|none)** — указывается подход к переменным, объявленным вне структурного блока. По умолчанию используется значение **shared** — все переменные, объявленные снаружи параллельной секции, будут общими для всех потоков, как объявленные через опцию **shared()**. Поэтому обычно опцию **shared()** можно не использовать явно. При значении **none** все внешние переменные, используемые внутри секции, но не указанные в опциях **private()**, **shared()**, **reduction()**, **firstprivate()**, **lastprivate()**, вызовут ошибку при компиляции.

8.2. Взаимодействие потоков. Директивы `critical`, `single`, `master`, `atomic`

Директивы предназначены для управления выполнением участков кода внутри параллельной секции.

Если нужно, чтобы участок кода выполнялся только одним потоком единовременно, то используется директива `#pragma omp critical`. В каждый момент времени код критической секции выполняется только одним потоком. Другие потоки будут заблокированы, пока один из них выполняет код. После завершения работы одного потока в секции один

из ожидающих потоков начинает выполнять код секции. Остальные продолжают ожидание. Обычно такой механизм используется, чтобы избежать ошибок (условий гонок, Race Condition) при работе с общими переменными. Однако использование последовательных участков кода в параллельной программе может снизить ее эффективность.

Директива `#pragma omp atomic` может использоваться перед одиночной конструкцией присваивания. Она гарантирует корректную работу с общей переменной в левой части присваивания. Для этого на время выполнения оператора присваивание всем потокам, кроме выполняющего операцию, блокируется доступ. При этом вычислениям, проводящимся в правой части, не гарантируется атомарность.

Если нужно, чтобы участок кода выполнялся только один раз, используется директива `#pragma omp single`. Какой именно поток будет выполнять структурный блок, не определено. Все остальные потоки будет ожидать завершения блока, если не указать опцию `nowait`.

Директива `#pragma omp master` выполняет структурный блок один раз, при этом только основным потоком. Все остальные потоки пропускают участок и продолжают работу со следующего фрагмента. Директива не осуществляет неявную синхронизацию.

8.3. Управление доступом к данным. Опции `private` и `shared`

В OpenMP переменные, используемые в параллельных областях, могут принадлежать к одному из двух классов:

- **private** (частные, приватные, локальные) — имеют свои независимые экземпляры в каждом потоке.

Поток может менять значение своих локальных переменных, не влияя на значение переменных с тем же именем в других потоках. Все переменные, объявляемые внутри параллельной секции, по умолчанию будут локальными.

- **shared** (общие) — существуют в единственном экземпляре и доступны всем потокам.

Если несколько потоков будут одновременно записывать значение общей переменной, не учитывая друг друга, не выполняя синхронизацию, то возникнет условие гонок — значение переменной и резуль-

тат выполнения программы будут непредсказуемы. По умолчанию все переменные, объявленные вне параллельной области, останутся общими при старте области.

Тип доступа к переменным можно явно указать в опциях директивы `#pragma omp parallel`.

8.4. Распределение работы между потоками. Директива `for`

В общем случае, для того чтобы добиться снижения времени выполнения программы, недостаточно просто создать параллельную область.

Рассмотрим следующую программу (см. Пример 2).

Очевидно, что самый трудозатратный участок — выполнение цикла. Для ускорения необходимо разделить работу между потоками.

OpenMP дает программисту несколько способов распределения работы между запущенными потоками. Большинство конструкций, предназначенных для распределения работы, не порождают новых потоков.

8.4.1. Ручное распределение работы

Один из вариантов распределения заключается в ручном разделении работы, при этом используются номера потоков. Разделим все итерации цикла на фрагменты, число которых равно количеству потоков. После завершения цикла сложим все промежуточные результаты в общую переменную (см. Пример 2а).

8.4.2. Автоматическое распараллеливание циклов

Директива `#pragma omp for` предназначена для автоматического распараллеливания выполнения цикла for. Компилятор автоматически распределит итерации цикла по потокам. При этом важно, чтобы полное число итераций было известно на момент входа в параллельную область.

Для краткости многие директивы могут быть использованы в объединенном виде, например, директива **parallel for** сразу создаст параллельные потоки и распределит части цикла по ним.

По умолчанию каждый поток приостановит работу после завершения структурного блока цикла. Программа продолжит работу, только когда все потоки завершат цикл (аналогично директиве **barrier**).

В стандарте OpenMP 2.0 возможно распараллеливание только для целочисленных счетчиков цикла. В OpenMP 3.0 распараллеливание возможно для итераторов стандартной библиотеки C++.

Опции:

- **private (<список переменных>)**;
- **firstprivate (<список переменных>)**;
- **reduction (<операция>:<переменная>)**;
- **lastprivate (<список переменных>)** — указывает, что у переменных будут индивидуальные копии в каждом потоке. После завершения блока, переменным будут присвоены значения из потока, завершившего цикл последним;
- **collapse (<число вложенных циклов>)** — указывает, что директива обрабатывает несколько тесно-вложенных циклов. Создается единое пространство итераций, которое распределяется по потокам. (директива *не поддерживается* в OpenMP 2.0.);
- **ordered** — указывает, что в теле цикла будет использоваться директива **ordered** (см. ниже);
- **nowait** — отменяет синхронизацию в конце цикла. Потоки продолжат работу, не дожидаясь завершения цикла во всех потоках;
- **schedule (<тип> [, <размер фрагмента>])** — задает способ распределения итераций цикла по потокам. Возможные типы распределения:
  - **static** — блочно-циклическое. Пространство итераций разделяется на непрерывные блоки заданного размера. Блоки распределяются по потокам последовательно: первый блок выполняется потоком 0, второй блок — потоком 1 и т. д. Если блоков больше, чем потоков, то распределение повторяется, начиная с потока 0. Если размер фрагмента не задан, то пространство итераций делится на блоки примерно равного размера, чтобы их количество совпадало с числом потоков;
  - **dynamic** — динамическое распределение с фиксированным размером блока. Итерации делятся на блоки заданного размера. Если размер не указан, то блоки будут состоять из одной итерации. Каждый поток получает новый блок из общей очереди сразу после выполнения предыдущего;
  - **guided** — динамическое распределение с уменьшающимся размером блока. Размер блока уменьшается с некоторого автоматически

выбранного значения, зависящего от числа потоков и общего числа итераций, до заданного значения. Это позволяет в ряде случаев лучше сбалансировать загрузку потоков;

– **auto** — способ распределения выбирается автоматически компилятором;

– **runtime** — способ распределения задается переменной окружения OMP_SCHEDULE.

Схемы распределения показаны на рис. 29.

Вычисление числа Пи реализуется с использованием директивы **parallel for** и опции **reduction** (см. Пример 2b).

8.5. Независимые параллельные секции. Директива sections

Директива `#pragma omp sections` позволяет использовать независимые структурные блоки кода, каждый из которых будет выполняться отдельным потоком.

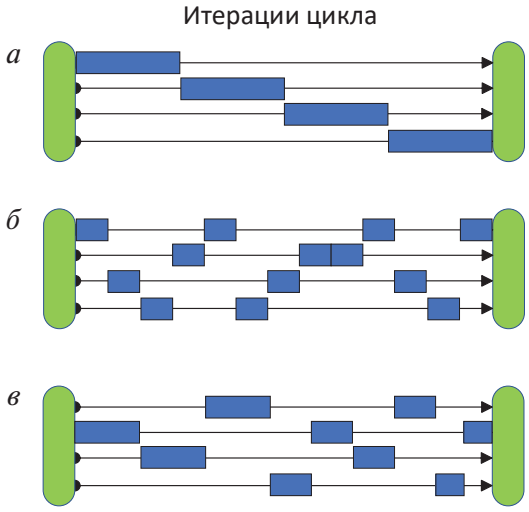


Рис. 29. Схемы распределения итераций цикла по потокам:

а — блочно-циклическое (static);
б — динамическое с фиксированным размером блока (dynamic);
в — динамическое с уменьшающимся размером блоков (guided)

Она поддерживает следующие опции:

- **private (<список переменных>)**;
- **firstprivate (<список переменных>)**;
- **reduction (<операция>:<переменная>)**;
- **lastprivate (<список переменных>)**;
- **nowait**.

Внутри блока `sections` используется директива `#pragma omp section` для задания структурного блока, выполняющегося одним потоком. Перед первым структурным блоком директива `section` необязательна. Если количество созданных потоков больше, чем количество блоков, определенных директивами `section`, то часть потоков не будет задействована.

Число структурных блоков-секций зафиксировано в коде, и ими нельзя управлять во время выполнения программы.

8.6. Выполнение независимых заданий. Директивы `task`, `taskwait`

Директива `#pragma omp task` используется для выделения независимой задачи. При достижении потоком структурного блока, обозначенного данной директивой, код блока заносится в очередь заданий. Такое задание может начать выполнение немедленно либо быть отложенным на неопределенное время.

Возможные опции директивы:

- **if (<условие>)**;
- **shared (<список переменных>)**;
- **private (<список переменных>)**;
- **firstprivate (<список переменных>)**;
- **default (shared|none)**;
- **untied** — опция обозначает задание, которое может быть выполнено любым потоком из активной параллельной секции. Если опция не указана, то задание может выполняться только тем потоком, который его породил.

Для синхронизации может быть использована директива `#pragma omp taskwait`. При ее достижении поток приостанавливает выполнение кода и ожидает завершения всех своих запущенных заданий.

Важным отличием заданий от механизма sections является возможность динамически запускать различное количество заданий в любое время.

8.7. Синхронизация. Директивы `barrier`, `ordered`

Самый простой способ синхронизации в OpenMP — использование барьера.

Директива `#pragma omp barrier` заставляет потоки приостановить работу. Выполнение программы продолжится, только когда *все* потоки достигнут данной директивы.

Директива `#pragma omp ordered` определяет внутри тела цикла участок кода, который будет выполнен в том же порядке, в котором идут итерации цикла. Директива может использоваться, например, при необходимости упорядоченного вывода.

8.8. Векторизация. Использование директив `simd` и `declare simd`

Векторная операция — такая операция, которая осуществляется над несколькими операндами одновременно. Использование векторных операций является частным случаем класса SIMD — одна команда обрабатывает несколько данных. Использование векторных операций может значительно ускорить вычисления.

Современные процессоры архитектуры x86-64 поддерживают различные наборы векторных расширений: MMX, SSE, AVX, AVX2, AVX-512.

Для автоматической векторизации циклов в OpenMP используется директива `#pragma omp simd`. Возможно совместное использование многопоточности и векторизации. Для этого используется гибридная директива `#pragma omp for simd`.

Необходимые требования к векторизуемому циклу:

- количество итераций должно быть известно заранее, либо во время компиляции, либо перед входом в цикл;

- выход из цикла осуществляется только после завершения всех итераций (нельзя использовать `break` или `goto`);
- в теле цикла не должны вызываться функции, кроме:
  - стандартных библиотечных функций (sin(), pow() и т. д.);
  - inline-функций;
  - функций, объявленных с помощью директивы `#pragma omp declare simd`.

Контрольные вопросы к разделу Б

1. Какие компиляторы языков C и C++ поддерживают технологию OpenMP?
2. В чем заключается модель многопоточности Fork-Join?
3. Чем отличается главный поток от подчиненных?
4. Какие ключи используются для компиляции OpenMP-программ?
5. Что произойдет, если запустить компиляцию OpenMP-программы без соответствующих ключей?
6. Как использовать Microsoft Visual Studio для компиляции программ OpenMP?
7. Какие команды используются в системе запуска задач SLURM?
8. Чем отличается интерактивный запуск программы от пакетного?
9. Как задать доступное программе количество ядер?
10. Как задается объем памяти, доступный программе?
11. Как выбрать раздел или узел для запуска программы?
12. Как перенаправлять потоки ввода и вывода в терминале?
13. Какие команда используются для просмотра списка запущенных задач?
14. Как отменить запущенную задачу?
15. Назовите основные компоненты технологии OpenMP.
16. Как использовать переменные окружения, чтобы задать количество потоков в программе?
17. Какие переменные окружения используются для привязки потоков к процессорам?
18. Как задать переменные окружения в терминале?
19. Как задавать переменные окружения в Visual Studio?
20. Какие директивы используются для создания и работы с параллельными областями?
21. Как использовать библиотечные процедуры OpenMP в языках C и C++?
22. Какие библиотечные процедуры используются для работы с потоками в OpenMP-программе?
23. Что вернет функция `omp_get_num_threads()`, если вызвать ее вне параллельной области?
24. Как измерить время выполнения фрагмента программы?
25. Какие опции могут использоваться в директиве `parallel`?

26. Для чего используется директива `critical`?
27. Чем отличаются директивы `single` и `master`?
28. Может ли главный поток выполнить блок, заданный директивой `single`?
29. Какие директивы используются для синхронизации потоков?
30. Для каких директив осуществляется неявная синхронизация?
31. Что произойдет, если использовать барьер так, что его достигнут не все потоки в параллельной области?
32. В чем разница между использованием критических секций и атомарных операций?
33. Как управлять доступом к переменным в многопоточной программе?
34. В чем разница между опциями `shared` и `private`?
35. Для чего предназначена опция `reduction`?
36. Как автоматически распределить работу цикла по потокам с помощью OpenMP?
37. Какие опции могут использоваться в директиве `for`?
38. Какими способами можно разделить итерации цикла по потокам?
39. Всегда ли при статическом распределении всем потокам достанется равное количество итераций?
40. При каких способах распределения одни и те же итерации могут при повторном запуске программы достаться другим потокам?
41. Можно ли распределить по потокам несколько вложенных циклов?
42. Какими способами можно организовать выполнение разного кода разными потоками в OpenMP?
43. В чем отличие механизмов секций и заданий?
44. Можно ли выполнять разными потоками различный код без использования директив `sections` или `task`?
45. В каких случаях может понадобиться механизм заданий?
46. Как использовать автоматическую векторизацию в OpenMP?
47. Какие циклы могут быть распараллелены и векторизованы?

Примеры программ к разделу Б

Пример 1. Последовательные и параллельные секции

В примере показано использование параллельных секций. Каждый поток выдает в консоль свой номер и общее количество потоков.

```
#include <iostream>
#include <omp.h>
int main ()
{
   std:: cout << "Hello, world from serial section" \
                                          << std:: endl;
#pragma omp parallel
   {
      int nthread = omp_get_num_threads();
      int thread_id = omp_get_thread_num();
      std:: cout << "Hello, world from thread "
         << thread_id << " of "
         << nthread << std:: endl;
   }
}
```

Компиляция и запуск:

```
g++ 1_helloworld.cpp -o 1hw -fopenmp
srun -c 8 ./1hw
```

Вывод программы:

```
Hello, world from serial section
Hello, world from thread 6 of 8
Hello, world from thread 2 of 8
Hello, world from thread 1 of 8
Hello, world from thread 4 of 8
Hello, world from thread 7 of 8
Hello, world from thread 0 of 8
```

```
Hello, world from thread 3 of 8
Hello, world from thread 5 of 8
```

Пример 2. Последовательная программа для вычисления числа Пи

Значение числа Пи может быть получено при помощи определенного интеграла:

$$\pi = \int_0^1 \frac{4}{1+x^2} dx.$$

Для численного интегрирования применяется метод прямоугольников.

```
#include <iostream>
#include <omp.h>

#define N 2000000000

int main ()
{
   double x;
   double sum = 0;
   double dx = 1.0 / (double) N;
   double start = omp_get_wtime();
   for (int i = 0; i < N; i++)
   {
      x = (i + 0.5) * dx;
      sum += 4.0 / (1.0 + x * x);
   }
   double end = omp_get_wtime();
   double pi = sum * dx;
   std:: cout << "Pi = " << pi << std:: endl;
   std:: cout << "time taken " << end – start \
                                        << std:: endl;
}
```

Компиляция и запуск:

```
g++ 2_pi.cpp -o 2pi -fopenmp
srun -c 1 ./2pi
```

Вывод программы:

```
Pi = 3.14159
time taken 19.5138
```

Пример 2а. Параллельная программа для вычисления числа Пи с ручным распределением данных

```
#include <iostream>
#include <omp.h>

#define N 2000000000

int main()
{
  double pi = 0;
  double dx = 1.0 / (double) N;
  double start = omp_get_wtime();
#pragma omp parallel
  {
    double x;
    double sum = 0;
    int chunk = N / omp_get_num_threads();
    int id = omp_get_thread_num();

    for (int i = id * chunk;
      i < std:: min (N, (id + 1) * chunk);
      i++)
    {
      x = (i + 0.5) * dx;
      sum += 4.0 / (1.0 + x * x);
    }
#pragma omp critical
    pi += sum * dx;
  }
  std:: cout << "Pi = " << pi << std:: endl;
  double end = omp_get_wtime();
```

```
   std:: cout << "time taken " << end – start \
                                        << std:: endl;
}
```

Запуск и вывод программы:

```
srun -c 1 ./2pi
Pi = 3.14159
time taken 15.5766

srun -c 8 ./2pi
Pi = 3.14159
time taken 2.52818
```

Пример 2b. Параллельная программа для вычисления числа Пи с использованием директивы parallel for

```
#include <iostream>
#include <omp.h>

#define N 2000000000

int main ()
{
   double x, pi, sum;
   double dx = 1.0 / (double) N;
   double start = omp_get_wtime();
#pragma omp parallel for private (x) reduction (+: sum)
   for (int i = 0; i < N; i++)
   {
      x = (i + 0.5) * dx;
      sum += 4.0 / (1.0 + x * x);
   }
   pi = sum * dx;
   std:: cout << "Pi = " << pi << std:: endl;
   double end = omp_get_wtime();
   std:: cout << "time taken " << end – start \
                                        << std:: endl;
}
```

Запуск и вывод программы:

```
srun -c 1 ./3pi_par
Pi = 3.14159
time taken 15.59

srun -c 8 ./3pi_par
Pi = 3.14159
time taken 2.66012
```

Пример 3. Вычисление гистограммы

Элементы массива инициализируются случайными целыми числами. Вычислить гистограмму получившегося массива, проверить верно ли работает параллельная программа.

```
#include <iostream>
#include <omp.h>

void fill_zero (long int* arr, long int count)
{
   int i;
   for (i = 0; i < count; i++)
    {
      arr [i] = 0;
   }
}

void fill_rand (long int* arr, long int count, \
   long int range)
{
   srand (time (NULL));
   int i;
   for (i = 0; i < count; i++)
   {
      arr [i] = rand()% (range);
   }
}

int main()
{
```

```
  constexpr long long Nhist = 500;
  constexpr long long Nupd = 1000000;
  long int* hist1, * hist2, * upd;
  hist1 = (long int*) malloc (Nhist * sizeof (long int));
  hist2 = (long int*) malloc (Nhist * sizeof (long int));
  upd = (long int*) malloc (Nupd * sizeof (long int));
  double time;
#pragma omp parallel
#pragma omp single
  std:: cout << "Thread count is: " << \
                omp_get_num_threads() << std:: endl;
  fill_zero (hist1, Nhist);
  fill_zero (hist2, Nhist);
  fill_rand (upd, Nupd, Nhist);
//Serial variant
  time = omp_get_wtime();
  for (int i = 0; i < Nupd; i++)
  {
    hist1 [upd [i]] ++;
  }
  time = omp_get_wtime() - time;
  std:: cout << "Serial time: " << time << std:: endl;
//Parallel variant
  time = omp_get_wtime();
#pragma omp parallel for reduction (+: hist2 [: Nhist])
  for (int i = 0; i < Nupd; i++)
  {
    hist2 [upd [i]] ++;
  }
  time = omp_get_wtime() - time;
  std:: cout << "Parallel time: " << time << std:: endl;
//Checking if results are same
  bool Error = false;
  for (int i = 0; i < Nhist; i++)
    if (hist1 [i]!= hist2 [i])
      Error = true;
  if (Error)
```

```
      std:: cout << "Error!" << std:: endl;
   else
      std:: cout << "OK!" << std:: endl;
   free (hist1);
   free (hist2);
   free (upd);
   return 0;
}
```

Пример 4. Вычисление простых чисел на заданном отрезке

Отрезок разделяется на фрагменты, и каждый поток вычисляет деление с остатком для чисел своего фрагмента.

```
#include <iostream>
#include <omp.h>

int main()
{
   constexpr long long N = 100000;
   double time;
#pragma omp parallel
#pragma omp single
   std:: cout << "Threads count is: " << \
      omp_get_num_threads() << std:: endl;

//Serial variant
   int num_prime = 0;
   time = omp_get_wtime();
   for (int i = 2; i <= N; i++)
   {
      int div = 0;
      for (int j = 2; j <= i; j++)
         if (i % j == 0)
            div++;
      if (div == 1)
         num_prime++;
   }
```

```
   time = omp_get_wtime () – time;
   std:: cout << "Serial time: " << time << std:: endl;
   std:: cout << "Prime numbers: " << num_prime << \
                                          std:: endl;
//Parallel variant
   num_prime = 0;
   time = omp_get_wtime();
#pragma omp parallel for reduction (+: num_prime) \
                                       schedule (guided)
   for (int i = 2; i <= N; i++)
   {
      int div = 0;
      for (int j = 2; j <= i; j++)
         if (i % j == 0)
            div++;
      if (div == 1)
         num_prime++;
   }
   time = omp_get_wtime() – time;
   std:: cout << "Parallel time: " << time << std:: endl;
   std:: cout << "Prime numbers: " << num_prime << \
                                          std:: endl;
   return 0;
}
```

Пример 5. Вычисление суммы двух векторов

Программа содержит три варианта реализации: без распараллеливания, распараллеливание с использованием OpenMP и гибридное распараллеливание с многопоточностью и векторизацией.

```
#include <iostream>
#include <omp.h>

float vSumm (float* A, float* V, float* C, int n)
{
   double start = omp_get_wtime();
   for (int i = 0; i < n; i++)
      C [i] = A [i] + V [i];
```

```
   return (float) (omp_get_wtime() - start);
}
float vSummOpenMP (float* A, float* V, float* C, int n)
{
   int i;
   double start = omp_get_wtime();
#pragma omp parallel for
      for (i = 0; i < n; i++)
        C [i] = A [i] + V [i];
   return (float) (omp_get_wtime() - start);
}
   float vSummOpenMPSIMD (float* A, float* V, float* C, \
                                                  int n)
{
   int i;
   double start = omp_get_wtime();
#pragma omp parallel for simd
      for (i = 0; i < n; i++)
        C [i] = A [i] + V [i];
   return (float) (omp_get_wtime() - start);
}


int main (int argc, char** argv) {
   constexpr long long N = 2000000000;
   float* A = new float [N];
   float* B = new float [N];
   float* C = new float [N];
   srand (time (NULL));

   for (int i = 0; i < N; i++) {
     A [i] = rand ()% 20000-10000;
     B [i] = rand ()% 20000-10000;
   }

   std:: cout << "Time for executing vSumm() " \
     << vSumm (A, B, C, N) << " seconds" \
     << std:: endl;
   std:: cout << "Time for executing vSummOpenMP() " \
     << vSummOpenMP (A, B, C, N) << " seconds" \
```

```
      << std:: endl;
   std:: cout << "Time for executing vSummOpenMPSIMD() "\
      << vSummOpenMPSIMD (A, B, C, N) << " seconds" \
      << std:: endl;

   delete A;
   delete B;
   delete C;

   return 0;
}
```

Пример 6. Вычисление произведения матрицы на вектор

Программа содержит три варианта реализации: без распараллеливания, распараллеливание с использованием OpenMP и гибридное распараллеливание с многопоточностью и векторизацией.

```
#include <iostream>
#include <omp.h>

float vmMult (float** A, float* V, float* C, int n)
{
   double start = omp_get_wtime();
   for (int j = 0; j < n; j++)
   {
      double sum = 0;
      for (int i = 0; i < n; i++)
         sum += A [j] [i] * V [i];
      C [j] = sum;
   }
   return (float) (omp_get_wtime() – start);
}

float vmMultOpenMP (float** A, float* V, float* C, int n)
{
   double start = omp_get_wtime();
#pragma omp parallel for
   for (int j = 0; j < n; j++)
```

```
   {
      double sum = 0;
      for (int i = 0; i < n; i++)
         sum += A [j] [i] * V [i];
      C [j] = sum;
   }
   return (float) (omp_get_wtime() — start);
}

float vmMultOpenMP_SIMD (float** A, float* V, float* C, \
                                                  int n)
{
   double start = omp_get_wtime();
#pragma omp parallel for
   for (int j = 0; j < n; j++)
   {
      double sum = 0;
#pragma omp simd reduction (+: sum)
      for (int i = 0; i < n; i++)
         sum += A [j] [i] * V [i];
      C [j] = sum;
   }
   return (float) (omp_get_wtime() — start);
}

int main (int argc, char** argv)
{
   constexpr long long N = 60000;

   float** A = new float* [N];
   for (int j = 0; j < N; j++)
      A [j] = new float [N];
   float* B = new float [N];
   float* C = new float [N];

   srand (time (NULL));

   for (int i = 0; i < N; i++)
   {
      B [i] = rand ()% 20000-10000;
```

```
      for (int j = 0; j < N; j++)
      {
        A [j] [i] = rand()% 20000-10000;
      }
   }

   std:: cout << "Time for executing vmMult() = " << \
              vmMult (A, B, C, N) << " seconds" << \
                                          std:: endl;
   std:: cout << "Time for executing vmMultOpenMP() = " \
         << vmMultOpenMP (A, B, C, N) << " seconds" \
                                       << std:: endl;
   std:: cout << "Time for executing vmMultOpenMP_SIMD()\
   = " << vmMultOpenMP_SIMD (A, B, C, N) << " seconds" \
                                       << std:: endl;

   delete A;
   delete B;
   delete C;

   return0;
}
```

Задания для самостоятельной работы к разделу Б

Площадь многоугольника

Составить программу, вычисляющую площадь многоугольника по заданному списку координат вершин. Распараллелить и ускорить ее с помощью OpenMP.

Площадь простого (без самопересечений) многоугольника с n вершинами, заданными координатами, можно найти, используя формулу площади Гаусса, также известную как формула шнурования (Shoelace formula):

$$S = \frac{1}{2}\left|\sum_{i=1}^{n-1}\left(x_i y_{i+1} - x_{i+1} y_i\right) + x_n y_1 - x_1 y_n\right|.$$

Эта формула напрямую следует из теоремы Грина, связывающей криволинейный интеграл по замкнутой кривой и двойной интеграл по области, ограниченной данной кривой. Схема формулы показана на рис. 30.

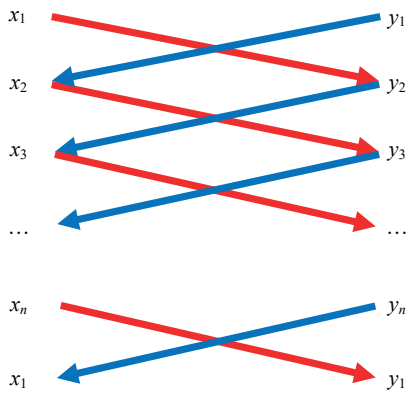


Рис. 30. Схема формулы шнурования:
красные стрелки — произведения со знаком «плюс»,
синие стрелки — произведения со знаком «минус»

Поиск палиндромов

Составить программу, выполняющую поиск всех подстрок-палиндромов в заданном тексте. Распараллелить и ускорить ее с помощью OpenMP.

Строка-палиндром — такая строка, которая читается одинаково как справа налево, так и слева направо. Поиск палиндромов в строке длиной n — одна из классических задач в компьютерных науках. Наивный алгоритм (рис. 31), псевдокод которого представлен ниже, имеет вычислительную сложность $O\left(n^2\right)$.

```
Find_Palindromes_Naive (string S)
{
// Между всеми символами, а также в начале и в конце,
добавляем специальный символ (например, `|`)
// Это необходимо для сведения всех палиндромов к
нечетной длине
   string S' = Modify (S, `|`)
// Инициализируем массив, в котором будут храниться
радиусы палиндромов вокруг каждой позиции в S'
   array PalindromeRadii [length (S')] = {0}
// Заметим:
//length (S') =length (PalindromeRadii) = 2 × length
(S) + 1
   for (Center = 1; Center < length (S')–1; Center++)
   {
// Ищем длиннейший палиндром, начинающийся в позиции
Center
      Radius = 0
      while Center- (Radius+1) >= 0 and \
         Center+ (Radius+1) < length (S') and \
      S' [Center- (Radius+1)] = S» [Center+ (Radius+1)]
      {
         Radius++
      }
```

```
// Сохраняем найденный радиус в массив
      PalindromeRadii [Center] = Radius
   }
// Сохраняем найденные палиндромы (длиннее 1
символа) в список, выбрасывая специальный символ
   list Palindromes = {}
   for (Center = 1; Center < length (S')-1; Center++)
   {
      Radius = PalindromeRadii [Center]
      if Radius > 1
      {
      Palindromes.append (UnModify (                \
                                 S' [Center – Radius: \
                                 Center + Radius + 1]))
      }
   }
   return Palindromes
}
```

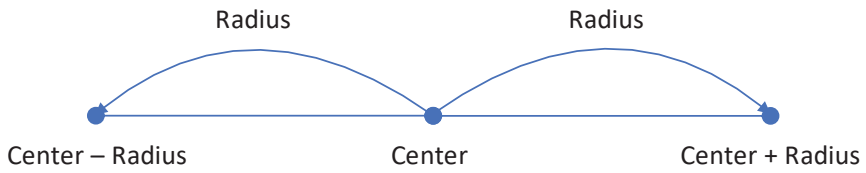


Рис. 31. Схема работы алгоритма поиска палиндромов в строке

Существует оптимизированный алгоритм для поиска всех подстрок-палиндромов, известный как алгоритм Манакера (Manacher's algorithm). Его сложность линейная, т. е. $O(n)$.

Задача Дирихле для уравнения Пуассона

Составить программу, решающую краевую задачу Дирихле для двумерного уравнения Пуассона. Распараллелить и ускорить ее с помощью OpenMP.

Рассматривается задача Дирихле для уравнения Пуассона:

$$\Delta u(x,y) = f(x,y), \quad u(x,y)\big|_{\partial D} = g(x,y).$$

Применяя метод конечных разностей с использованием пятиточечного шаблона (рис. 32) на сетке $N \times N$, реализовать параллельный итерационный процесс одним из следующих методов:

Гаусса — Зейделя для уравнения Пуассона в конечно-разностной форме:

$$u_{ij}^{k+1} = 0.25(u_{i-1,j}^{k+1} + u_{i+1,j}^{k} + u_{i,j-1}^{k+1} + u_{i,j+1}^{k} - h^2 f_{ij}); \quad i,j = 1,\ldots,N-1$$

Параллельный метод верхней релаксации по схеме:

$$u_{ij}^{k+1} = \frac{\omega}{4}(u_{i-1,j}^{k+1} + u_{i+1,j}^{k} + u_{i,j-1}^{k+1} + u_{i,j+1}^{k} - h^2 f_{ij}) + (1-\omega)u^{k}_{ij}; \quad i,j = 1,\ldots,N-1,$$

где ω – параметр верхней релаксации, например, $\omega = 1/2$ или $\omega = 1/4$ (при $\omega = 1$ получаем метод Гаусса — Зейделя).

В качестве области задания D функции $u(x,y)$ будем использовать единичный квадрат:

$$D = \{(x,y) \in D:\ 0 \le x,y \le 1\}.$$

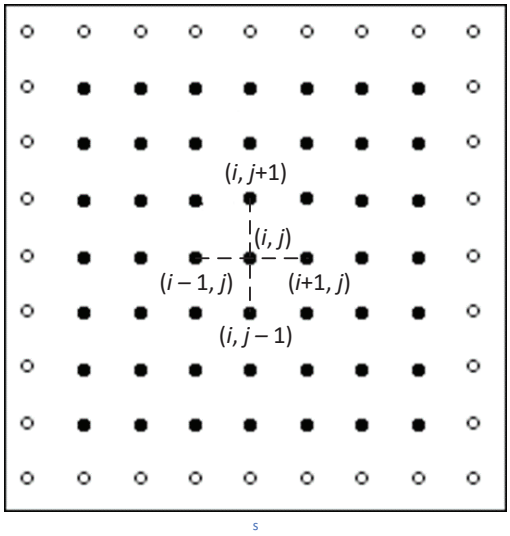


Рис. 32. Пятиточечный шаблон
на двумерной сетке

Граничные условия и правая часть, например, задаются в виде:

$$f(x,y) = 4 + 2x^2 - 2x + 2y^2 - 2y, \ \ (x,y) \in D;$$

$$u(x,y)\big|_{\partial D} = y^2 - y, \ \ x = 0, \ \ x = 1;$$
$$u(x,y)\big|_{\partial D} = x^2 - x, \ \ y = 0, \ \ y = 1.$$

В качестве начального приближения u_{ij} будем использовать значения, сгенерированные датчиком случайных чисел из диапазона $[-1,1]$.

Итерационный процесс продолжается до достижения заданной точности, а именно:

$$\max_{ij=1,\ldots,N} \left\{ \left| u_{ij}^{k+1} - u_{ij}^{k} \right| \right\} < \varepsilon, \quad \varepsilon = 0.01, \quad N = 1000.$$

Точное решение задачи имеет вид:

$$u(x,y) = (x^2 - x + 1)(y^2 - y + 1).$$

После решения задачи необходимо построить функцию u_{ij} на сетке с помощью программы Matlab/Octave и сравнить приближенное решение с точным.

Для построения параллельного алгоритма решения задачи необходимо провести разбиение исходной расчетной области на подобласти, количество которых соответствует количеству задействованных в расчете потоков. Для решения задачи в каждой области необходимы граничные условия. При этом учитывая шаблон разностной схемы, потребуется организация синхронизации для получения правильных значений приближений решения в узлах, являющихся границами организованных подобластей.

Значения функции на границах должны быть получены из соседних областей. Таким образом, все потоки должны на каждой итерации синхронизироваться, чтобы считать правильные значения. Счет не должен продолжаться, пока соседние потоки не запишут значения функции в память. Синхронизация обеспечивается директивой `#pragma omp barrier`.

Кроме того, на каждой итерации необходимо проверять, не достигнута ли нужная точность. Это возможно благодаря использованию опции **reduction** в цикле расчета значения невязки, в данном случае необходимо сложение значений, полученных потоками.

Заметим, что метод верхней релаксации представляет собой рекуррентные вычислительные формулы, затрудняющие его распараллеливание. Тем не менее метод все же удается распараллелить. Для этого необходимо разбить все узлы в шахматном порядке на красные и черные, как показано на рис. 33 (в литературе эта процедура носит название красно-черного или шахматного упорядочения). Нетрудно заметить, что для каждого цвета узлов расчеты будут проводиться независимо. На первом этапе будем вычислять значения сеточной функции $u_{i,j}$ в красных узлах, используя значения в черных узлах на предыдущей итерации:

$$u_{R,i,j}^{k+1} = \frac{\omega}{4}\left(u_{B,i-1,j}^{k} + u_{B,i+1,j}^{k} + u_{B,i,j-1}^{k} + u_{B,i,j+1}^{k}\right) + (1-\omega)u_{R,i,j}^{k} \ .$$

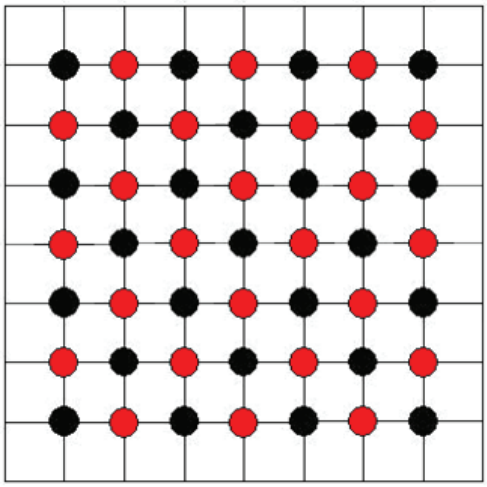


Рис. 33. Красно-черное разбиение узлов двумерной сетки

На втором этапе можно рассчитать значения $u_{i,j}$ в черных узлах, используя значения в красных узлах, но уже новые, вычисленные на первом этапе:

$$u_{B,i,j}^{k+1} = \frac{\omega}{4}\left(u_{R,i-1,j}^{k} + u_{R,i+1,j}^{k} + u_{R,i,j-1}^{k} + u_{R,i,j+1}^{k}\right) + (1-\omega)u_{B,i,j}^{k} \ .$$

Список рекомендуемой литературы

1. Акимова, Е. Н. Параллельные вычисления : учебное пособие / Е. Н. Акимова ; под общ. ред. П. С. Мартышко. — Екатеринбург : Изд-во УГТУ-УПИ, 2007. — 84 с. — ISBN 978-5-321-01158-4.

2. Антонов, А. С. Параллельное программирование с использованием технологии OpenMP / А. С. Антонов. — Москва : Изд-во МГУ, 2009. — 78 с. — ISBN 978-5-211-05702-9.

3. Воеводин, В. В. Параллельные вычисления : учебное пособие / В. В. Воеводин, Вл. В. Воеводин. — Санкт-Петербург : БХВ-Петербург, 2002. — 608 с. — ISBN 5-94157-160-7.

4. Гергель, В. П. Основы параллельных вычислений для многопроцессорных вычислительных систем : учебное пособие / В. П. Гергель, Р. Г. Стронгин. — 2-е изд., доп. — Нижний Новгород : Изд-во ННГУ, 2003. — 184 с. — ISBN 5-85746-602-4.

5. Ортега, Дж. Введение в параллельные и векторные методы решения линейных систем / Дж. Ортега ; перевод с английского Х. Д. Икрамова, И. Е. Капорина ; под редакцией Х. Д. Икрамова. — Москва : Мир, 1991. — 366 с. — ISBN 5-03-001941-3.

6. Самарский, А. А. Методы решения сеточных уравнений / А. А. Самарский, Е. С. Николаев. — Москва : главная редакция физико-математической литературы издательства «Наука», 1978. — 592 с.

7. Таненбаум, Э. Архитектура компьютера ; [перевод с английского Е. Матвеева] / Э. Таненбаум, Т. Остин. — Санкт-Петербург : Питер, 2013. — 816 с. — ISBN 978-5-496-00337-7.

8. Hwang, K. Computer Architecture and Parallel Processing / K. Hwang, F. A. Briggs. — New York : McGraw-Hill, 1984. — 846 p. — ISBN 9781259029141

9. OpenMP : [сайт]. — URL: http://openmp.org/ (дата обращения: 01.12.2021)

10. Что такое OpenMP? // Лаборатория Параллельных информационных технологий НИВЦ МГУ : [сайт]. — URL: https://parallel.ru/tech/tech_dev/openmp.html (дата обращения 01.12.2021).

11. Параллельные вычисления в УрО РАН : [сайт]. — URL: https://parallel.uran.ru (дата обращения 01.12.2021).

12. Intel 64 and IA-32 Architectures Optimization Reference Manual. — URL: https://hpc.icc.ru/hardware/64-ia-32-architectures-optimization-manual.pdf (дата обращения 13.01.2022).

Учебное издание

Акимова Елена Николаевна
Мисилов Владимир Евгеньевич

Параллельные вычисления на суперкомпьютерах с помощью технологии OpenMP

Редактор *С. А. Курмышкина*
Корректура и дизайн *Ю. В. Ершовой*

Подписано в печать 28.03.2023. Формат 70×100 1/16
Бумага писчая. Цифровая печать. Усл. печ. л. 8,4.
Уч.-изд. л. 5,2. Тираж 30 экз. Заказ № 22.

Издательство Уральского университета
Редакционно-издательский отдел ИПЦ УрФУ
620049, Екатеринбург, ул. С. Ковалевской, 5
Тел. 8 (343) 375-48-25, 375-46-85, 374-19-41
E-mail rio@urfu.ru

Отпечатано в Издательско-полиграфическом центре УрФУ
620075, Екатеринбург, ул. Тургенева, 4
Тел.: 8 (343) 350-56-64, 350-90-13
Факс: 8 (343) 358-93-06
E-mail: press-urfu@mail.ru

Для заметок

Для заметок


ПИЛОТ УНИВЕРСИТЕТСКОГО КНИГОИЗДАНИЯ
УНИВЕРСИТЕТСКАЯ КНИГА
2020


9 785799 636296